L1 Week 1: k3s on WSL2 + Argo CD

Cluster **halo** · Checkpoint: Argo CD syncing a sample app from Git

Production-style home-lab build on the Minisforum MS-S1 Max (Windows 11 + WSL2 Ubuntu 24.04)

k3s v1.36.5+k3s1 · kubectl v1.36.6 · Helm v4.3.1 · Argo CD chart 10.11.6 (v3.5.6) · podinfo 6.17.0

Versions checked 2026-11-02 · Project MLOps / ai-platform-portfolio

How to copy commands: every command box is numbered and colour-coded by where it runs. Select the whole box (from its first to last line) and paste it into the matching shell.

WSL Ubuntu shell (bash) **Windows PowerShell** **Output / reference - not a command**

Text tags: [WIN-ADMIN] = PowerShell as Administrator · [WIN] = normal PowerShell · [WSL] = WSL shell (`wsl -d halo`) · [BROWSER] = browser · [GITHUB] = GitHub web UI · ☐ = checkbox to tick

Contents

Overview: versions, architecture, plan

- Version table (checked 2026-11-02)
- Assumptions (I went with defaults; each one has a check in Step 0)
- Architecture
- Time plan (90-minute blocks, each one ends in a working state)
- Conventions used in every step
- Sources and references

Step 0: Windows + WSL2 foundation

- 0.1 Windows and BIOS checklist (B1)
- 0.2 Install / update WSL and Ubuntu 24.04 (B1)
- 0.3 .wslconfig: size, networking, keep-alive (B1)
- 0.4 /etc/wsl.conf: systemd, hostname, PATH hygiene (B1)
- 0.5 Kernel and networking prerequisites for k3s (B2)
- 0.6 Keep it running: idle timeouts, logon task, no sleep (B2)
- 0.7 Filesystem layout and disk hygiene (B2)
- 0.8 GPU flag (information only, don't set up now)
- 0.9 First baseline snapshot (B2)

Steps 1-3: k3s, client tools, verification

- Step 1: Install k3s (single node, pinned) (B3)
- Step 2: Client tools in WSL and on Windows (B4)
- Step 3: Verify the cluster end to end (B4)

Step 4: Monorepo + GitOps layout

- 4.1 Create the repo and your Git identity
- 4.2 Skeleton, placeholders, .gitignore, .gitattributes
- 4.3 ADR-0001
- 4.4 Capture the k3s host config as code (platform/k3s/halo/)
- 4.5 The GitOps layout (platform/kubernetes/)
- 4.6 Check for secrets, commit, push

Steps 5-7: Argo CD, app of apps, podinfo

- Step 5: Install Argo CD with Helm (B6)
- Step 6: AppProjects + the root Application (app of apps) (B7)
- Step 7: Sample app podinfo: base + halo overlay, deployed only through Git (B7)

Steps 8-10: Prove GitOps, rebuild, self-manage

- Step 8: Prove GitOps works (B8)
- Step 9: Rebuild drill (B9)
- Step 10 (optional, B10): Argo CD manages its own Helm install

Appendix: troubleshooting, glossary, checklist

- Troubleshooting
- Glossary
- Final checklist: "Argo CD syncing a sample app from Git"
- What's next (out of scope this week)

L1 · Week 1: k3s on WSL2 + Argo CD (cluster halo)

Checkpoint: *Argo CD syncing a sample app from Git.* **Framing:** a production-style platform built on one box. Every choice is the one a large organisation would make, scaled down. The end goal is to self-host several LLMs on Kubernetes so you stop paying per token.

Part	File	Steps	Blocks
Overview (this page)	`README.md`	versions, architecture, plan, assumptions	—
Windows + WSL foundation	`01-windows-wsl.md`	Step 0	B1, B2
k3s, clients, verification	`02-k3s-cluster.md`	Steps 1–3	B3, B4
Monorepo + GitOps layout	`03-monorepo-gitops-layout.md`	Step 4	B5
Argo CD, app of apps, podinfo	`04-argocd-app-of-apps.md`	Steps 5–7	B6, B7
Prove GitOps, rebuild, self-manage	`05-prove-rebuild.md`	Steps 8–10	B8, B9, (B10)
Troubleshooting, glossary, final checklist, what's next	`06-appendix.md`	—	—

Version table (checked 2026-11-02)

Component	Pinned version	Released	Source
WSL (Store package)	**3.0.5**	2026-10-29	https://github.com/microsoft/WSL/releases
Ubuntu WSL image	**Ubuntu-24.04** → 24.04.5	24.04.5 point release	https://github.com/microsoft/WSL/blob/master/distributions/DistributionInfo.json
WSL kernel (info only, ships with WSL)	6.18.x	—	https://github.com/microsoft/WSL2-Linux-Kernel
k3s	**v1.36.5+k3s1**	2026-09-24	https://github.com/k3s-io/k3s/releases/tag/v1.36.5%2Bk3s1
kubectl	**v1.36.6**	—	https://dl.k8s.io/release/stable-1.36.txt
Helm	**v4.3.1**	2026-10-14	https://github.com/helm/helm/releases/tag/v4.3.1
Argo CD Helm chart `argo/argo-cd`	**10.11.6** (appVersion **v3.5.6**)	2026-10-26	https://github.com/argoproj/argo-helm/releases/tag/argo-cd-10.11.6
argocd CLI	**v3.5.6**	2026-10-20	https://github.com/argoproj/argo-cd/releases/tag/v3.5.6
Sample app image	**ghcr.io/stefanprodan/podinfo:6.17.0**	2026-10-26	https://github.com/stefanprodan/podinfo/releases/tag/6.17.0
busybox (PVC smoke test)	**busybox:1.37.0**	—	https://hub.docker.com/_/busybox
k9s (optional)	**v0.51.0**	2026-06-05	https://github.com/derailed/k9s/releases/tag/v0.51.0

Component	Pinned version	Released	Source
Traefik, CoreDNS, local-path, metrics-server	bundled with the k3s release (not pinned separately)	—	k3s release notes (link above)

Why these versions - **k3s v1.36.5, not v1.37.1.** 1.37 is the newest minor. Production teams run n-1, and k3s's own `stable` channel usually tracks n-1. The CKA usually runs one or two minors behind upstream (most likely 1.35 or 1.36 right now). Check the version for your exam date on the CNCF CKA page. kubectl 1.36.6 is within the supported ±1 minor skew. - **Helm 4** is current (Helm 3 only gets security fixes). For you, the main change is that new installs use server-side apply by default. Every `helm` command in this guide works the same way on Helm 4. - **Argo CD 3.x.** Changes since 2.x that matter here: resource tracking uses annotations by default, and RBAC for logs and fine-grained update/delete is enforced. The chart is now at major version 10. - **WSL 3.0.x.** Every `.wslconfig` key used here was checked against the WSL source at that version.

Caveat: my build sandbox couldn't reach `update.k3s.io` or `argoproj.github.io`, so I took versions from the Git tags and the chart's `Chart.yaml`. Run this 30-second re-check in WSL once Step 0 is done:

#001 WSL Ubuntu shell (bash) - copy & paste

```
curl -s https://update.k3s.io/v1-release/channels | jq -r '.data[] | select(.id=="stable" or .id=="v1.36") | "\(.id) \(.latest)"'
helm repo add argo https://argoproj.github.io/argo-helm >/dev/null; helm repo update >/dev/null; helm search repo argo/argo-cd --versions | head -3
```

If `v1.36` shows a newer patch than v1.36.5+k3s1, you can use it. Change the version in one place: `platform/k3s/halo/install-k3s.sh`.

Assumptions (I went with defaults; each one has a check in Step 0)

#	Assumption	Where it's checked or changed
A1	Windows 11 23H2 or later. Nothing here needs mirrored mode, so build number matters little	Step 0.1
A2	No Docker Desktop, or you remove its `kubectl` from PATH	Step 0.1 (check + fix)
A3	WSL gets **48 GB RAM, 16 logical CPUs, 16 GB swap**. Shrink it if your BIOS gives the GPU a big share of RAM (table in Step 0.3)	Step 0.3
A4	The WSL disk stays on C:. Use `--location D:\WSL\halo` if you have a second NVMe	Step 0.2
A5	GitHub user `nikhilgs817` (inferred from your other repo). If it's different, replace it everywhere with one `sed` (Step 4)	Step 4
A6	The PC doesn't sleep on AC power (Phoenix has to run around the clock)	Step 0.6
A7	The guide lives in the `certifiaction` repo. You type the files into `ai-platform-portfolio` yourself	—

Architecture

Diagram

```
Windows 11 (daily PC) - MS-S1 Max, 128 GB, Radeon 8060S
 |  Browser ...... http://podinfo.halo.localhost  |  http://argocd.halo.localhost   (*.localhost -> 127.0.0.1, no DNS)
 |  PowerShell ... kubectl / helm / argocd -> kubeconfig https://127.0.0.1:6443
 |  Task Scheduler at logon -> keeps WSL distro "halo" alive
 |
 '--> WSL localhost relay (NAT mode, localhostForwarding): 127.0.0.1:80 / :443 / :6443  --> into the VM
        |
        v
 WSL2 utility VM (Hyper-V) - 48 GB / 16 vCPU
 '-- Ubuntu 24.04 distro "halo" (systemd, ext4.vhdx)
      |-- eth0   172.x.x.x       (changes every boot - not used by k3s)
      |-- halo0  10.255.255.10   (dummy NIC, stable node IP)
      '-- k3s v1.36.5 server (systemd), node "halo"
           |-- control plane: apiserver, scheduler, controller-manager, embedded etcd, containerd, flannel
           |-- Traefik (hostNetwork :80/:443)
           |     |-- Host: argocd.halo.localhost   --> Service argocd-server  (ns argocd)
           |     '-- Host: podinfo.halo.localhost  --> Service podinfo --> pods  (ns podinfo)
           |-- Argo CD --(apply / prune / self-heal)--> podinfo
           '-- local-path PVs: /var/lib/rancher/k3s/storage
                |
 Argo CD  --(git pull over SSH every ~3 min, read-only deploy key)-->  GitHub: ai-platform-portfolio (private)
```

Network path from a Windows browser to the app: `podinfo.halo.localhost` → the browser resolves `*.localhost` to 127.0.0.1 itself (no DNS involved) → Windows `127.0.0.1:80` → WSL localhost relay → Traefik bound on `:80` in the distro's network namespace (hostNetwork) → Ingress rule matches `Host` → Service `podinfo` (ClusterIP) → pod IP on flannel `10.42.0.0/16`.

Three design decisions everything else depends on

1. **NAT networking + a stable dummy-interface node IP (recommended), not mirrored.**
   - **NAT** gives the distro one simple `eth0` on a private 172.x subnet, and its IP changes on every boot. k3s doesn't care, because the node is pinned to `halo0` = `10.255.255.10`, a dummy interface that never changes. Windows reaches the cluster through *localhost forwarding*. That forwarding only relays real listening sockets, which is why Traefik binds ports 80/443 directly (hostNetwork) instead of through ServiceLB's iptables rules.
   - **Mirrored** mode copies all of Windows' adapters (Wi-Fi, Ethernet, VPN) into Linux. k3s would then auto-detect whichever one is active, so the node IP changes when you switch networks. You'd also get IPv6 `::1` quirks and Hyper-V firewall rules on inbound traffic. You'd gain only one

thing: other LAN devices (your phone viewing the Phoenix dashboard) could reach the cluster directly. Because the node IP is pinned, switching to mirrored later is a one-line `.wslconfig` change. Defer that decision.

2. **Embedded etcd (`cluster-init: true`), not k3s's default SQLite.** This matches production and lets you practise CKA etcd snapshot and restore. etcd hates IP changes, and the stable dummy IP is what makes it safe in WSL.
3. **One root Application per cluster (`clusters/halo`) plus Kustomize overlays per environment.** The future `aws` cluster gets its own root (`clusters/aws`) and its own overlays, and nothing has to move.

Lab vs production (what this box simplifies, and what replaces it later)

Concern	halo (this week)	Large-org production	When you close the gap
Control plane	1 node, embedded etcd	3 control-plane nodes + workers, HA etcd	CKA kubeadm practice; `aws` cluster
Ingress edge	Traefik hostNetwork on one node	Cloud LB → ingress controller fleet	`aws` overlay
TLS	none (http on `*.localhost`)	cert-manager + internal CA / ACME	after week 1
Argo CD auth	local `admin`, password in password manager	SSO (OIDC), `admin.enabled: false`, RBAC per team	later
Git access	read-only deploy key	GitHub App with org-scoped permissions	later
Secrets	created by CLI, never in Git	Sealed Secrets / External Secrets + Vault / AWS SM	after week 1
Images	public registries	Harbor proxy-cache + Trivy + cosign + Kyverno	L1 weeks 2–4
Bootstrap	2 shell scripts in Git	Terraform + Ansible + Argo CD bootstrap	Terraform for AWS
Storage	local-path (single disk, no replication)	CSI with replication/snapshots (EBS, Longhorn, Ceph)	Phoenix: backups first

---

Time plan (90-minute blocks, each one ends in a working state)

Block	Steps	Ends with (working state)
B1	0.1–0.5 BIOS/Windows checks, WSL install, `.wslconfig`, `wsl.conf`	`wsl -d halo` opens a shell, hostname `halo`, systemd running
B2	0.6–0.9 kernel prerequisites, keep-alive task, sleep settings, **baseline export**	`halo-baseline.tar` safe on disk
B3	1 k3s install (stable IP, etcd, Traefik config)	`kubectl get nodes` → `halo Ready` inside WSL
B4	2–3 client tools on both sides, smoke tests, restart test	Browser on Windows opens `hello.halo.localhost`; survives `wsl --shutdown`
B5	4 GitHub repo, monorepo skeleton, ADR-0001, GitOps folders	First commit pushed to `main`
B6	5 Argo CD via Helm, UI via Traefik, password rotated, repo connected	Argo CD UI reachable, repo status *Successful*
B7	6–7 AppProjects, root app, podinfo base + halo overlay	**Checkpoint hit:** podinfo deployed by Argo CD from Git
B8	8a–8f the six GitOps proofs	Each proof ticked, history visible

Block	Steps	Ends with (working state)
B9	9 rebuild drill + runbook	`docs/runbooks/rebuild-halo.md` with measured times
B10	*10 optional: Argo CD manages itself*	*Argo CD Application `argocd` Synced*

If you only get 6 blocks this week, stop after **B7**. That's the checkpoint. B8 and B9 can move to next week without leaving anything broken.

Conventions used in every step

- **Where to run** tags: [WIN-ADMIN] = Windows PowerShell **as Administrator** · [WIN] = Windows PowerShell (normal) · [WSL] = WSL Ubuntu shell (`wsl -d halo`) · [BROWSER] = browser · [GITHUB] = GitHub web UI.
- Commands are copy-paste. Files are written with `cat > path <<'EOF'` so you get the exact path and the full contents in one block.
- `- [ ]` boxes are for you to tick.
- Every major step ends with **CKA corner**, **Interview angle** and **Phoenix angle**.

Sources and references

- WSL configuration (`.wslconfig`, `wsl.conf`): https://learn.microsoft.com/windows/wsl/wsl-config
- WSL networking modes: https://learn.microsoft.com/windows/wsl/networking
- WSL disk management: https://learn.microsoft.com/windows/wsl/disk-space
- k3s configuration file and server flags: https://docs.k3s.io/installation/configuration · https://docs.k3s.io/cli/server
- k3s networking (Traefik, ServiceLB): https://docs.k3s.io/networking/networking-services
- k3s embedded etcd + snapshots: https://docs.k3s.io/datastore/ha-embedded · https://docs.k3s.io/cli/etcd-snapshot
- Argo CD declarative setup, projects, app of apps: https://argo-cd.readthedocs.io/en/stable/operator-manual/declarative-setup/ · https://argo-cd.readthedocs.io/en/stable/operator-manual/cluster-bootstrapping/
- Argo CD Helm chart: https://github.com/argoproj/argo-helm/tree/main/charts/argo-cd
- AMD ROCm on WSL (Ryzen / Strix Halo): https://rocm.docs.amd.com/projects/radeon-ryzen/en/latest/docs/install/installryz/wsl/howto_wsl.html · Phoronix, ROCDXG for WSL: https://www.phoronix.com/news/AMD-WSL-ROCm-ROCDXG

Step 0: Windows + WSL2 foundation (blocks B1–B2)

Outcome of this step: an Ubuntu 24.04 distro named `halo` with systemd, sized for the lab, that stays running after you close the terminal. The kernel prerequisites are verified, and a clean baseline snapshot is saved to disk.

0.1 Windows and BIOS checklist (B1)

Goal: confirm the machine can run WSL2 and has no conflicting tools. **Why:** WSL2 is a small Hyper-V VM, so hardware virtualization (AMD SVM) has to be on. Docker Desktop and older tools often put their own `kubectl.exe` on PATH, and later you'd be talking to the wrong cluster without noticing. Think of this as the VMware host-readiness check before installing ESXi. **Where:** [WIN-ADMIN]

#002 Windows PowerShell - copy & paste

```
# Windows version (any 23H2 / 24H2 / 25H2 build is fine)
[System.Environment]::OSVersion.Version; (Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion').DisplayVersion

# Virtualization: "A hypervisor has been detected" or "Virtualization Enabled In Firmware: Yes"
systeminfo | Select-String -Pattern 'Hyper-V|hypervisor'

# RAM Windows can actually see (Strix Halo hands part of the 128 GB to the GPU)
"{0:N1} GB visible to Windows" -f ((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory/1GB)

# Conflicts: other kubectl/helm on PATH, Docker Desktop distros, ports 80/443/6443 in use
Get-Command kubectl,helm,argocd -All -ErrorAction SilentlyContinue | Select-Object Source
wsl -l -v 2>$null
Get-NetTCPConnection -State Listen -LocalPort 80,443,6443 -ErrorAction SilentlyContinue |
  Select-Object LocalPort, OwningProcess, @{n='Process'; e={(Get-Process -Id $_.OwningProcess).ProcessName}}
```

Expected: a version like `10.0.26100` with DisplayVersion `24H2` (or newer); hypervisor detected or virtualization enabled; about 96–124 GB visible; no `kubectl` found; nothing listening on 80/443/6443.

- ☐ Virtualization is on. If not: reboot into BIOS (MS-S1 Max: **Del** at boot) → *Advanced → CPU Configuration → SVM Mode = Enabled*. Leave *IOMMU* on Auto. Save, then reboot.
- ☐ No stray `kubectl.exe`. If Docker Desktop is installed: *Settings → Kubernetes → off*, and remove `C:\Program Files\Docker\Docker\resources\bin` from PATH, or uninstall Docker Desktop. Docker Desktop's `docker-desktop` distro can stay.
- ☐ Ports 80/443/6443 are free on Windows. If `System` (PID 4) holds port 80, that's http.sys (IIS, or a service such as "World Wide Web Publishing"). Stop or disable it, or later change the Traefik ports (appendix T7).
- ☐ Wrote down the **visible RAM** number. You need it in 0.3.

If it fails 1. `systeminfo` says virtualization is disabled → turn on SVM in the BIOS (above). 2. Corporate-style device policy blocks WSL → this is your personal PC; check *Settings → Privacy & security → For developers* and make sure no MDM profile is applied.

0.2 Install / update WSL and Ubuntu 24.04 (B1)

Goal: WSL 3.0.x installed, and Ubuntu 24.04 registered as a distro named **halo**. **Why:** WSL now ships through the Microsoft Store, not as a Windows feature, so `wsl --update` is how you patch it, and the Linux kernel comes with it. `wsl --install --no-distribution` turns on the one Windows feature it needs (*Virtual Machine Platform*). The *Windows Subsystem for Linux* optional feature is only needed for WSL1. Naming the distro `halo` (with `--name`) means every command and backup refers to the cluster by name. **Where:** [WIN-ADMIN]

#003 Windows PowerShell - copy & paste

```
wsl --install --no-distribution     # enables Virtual Machine Platform + installs WSL
# >>> If it asks for a reboot, reboot now and reopen PowerShell as Administrator <<<
wsl --update
wsl --version                       # expect: WSL version: 3.0.5 (or newer), Kernel version: 6.18.x
wsl --set-default-version 2
wsl --list --online | Select-String 24.04   # expect: Ubuntu-24.04   Ubuntu 24.04 LTS
```

Write `.wslconfig` (0.3) **before** installing the distro. `sparseVhd` only applies to disks created after it is set.

Then install. Use the default location, or add `--location D:\WSL\halo` for a second drive:

#004 Windows PowerShell - copy & paste

```
wsl --install Ubuntu-24.04 --name halo
# First launch asks for a UNIX username + password. Use a short lowercase name; this password is your sudo password.
```

Expected output ends with: `Installing: Ubuntu 24.04 LTS … The operation completed successfully.` After you set a password you get a shell prompt like `nikhil@<random-hostname>:~$`. Type `exit`.

#005 Windows PowerShell - copy & paste

```
wsl --set-default halo
wsl -l -v      # expect:  * halo   Stopped/Running   2
```

Verify: `wsl -l -v` shows `halo` with VERSION `2` and an asterisk (default).

If it fails 1. `0x80370102` / "Please enable the Virtual Machine Platform" → SVM is off in the BIOS, or the feature isn't on yet: `dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart`, then reboot. 2. `--name` / `--location` "unknown option" → WSL is too old. Run `wsl --update` again, or install the latest `.msixbundle` from the WSL releases page. 3. The download stalls → `wsl --install Ubuntu-24.04 --name halo --web-download`.

0.3 `.wslconfig`: size, networking, keep-alive (B1)

Goal: give the WSL VM fixed limits that leave Windows comfortable, pick the network mode, and turn off the idle shutdown. **Why:** `.wslconfig` configures the *VM* (all distros), like the VM settings in vSphere: vCPU, RAM, swap, NIC type. Without limits, WSL can take up to half your RAM and holds on to the file cache. Strix Halo memory is *unified*: whatever the BIOS / AMD *Variable Graphics Memory* (VGM) gives the GPU disappears from Windows. Because your end goal is several local models, the GPU share will likely grow, so size WSL from what Windows *sees*, not from 128 GB.

Sizing rule: `WSL memory ≈ min(48 GB, visible RAM − GPU-heavy Windows work − 24 GB)`

VGM (GPU share)	Visible to Windows	Recommended WSL `memory`	Notes
≤ 16 GB	~112 GB+	**48GB**	Default for week 1
32 GB	~96 GB	**48GB**	Good balance
64 GB	~64 GB	**32GB**	GPU models on Windows side, platform in WSL
96 GB	~32 GB	16GB (tight)	Only if all models run on the GPU; drop the observability stack

Headroom budget (why 48 GB): k3s + etcd ~1.5 GB · Argo CD ~1 GB · Traefik 0.1 GB · Phoenix (Timescale 4 + Redis 1 + services 2) ~7 GB · observability (Prometheus/Loki/Tempo/Grafana) ~6–8 GB · Harbor ~4 GB · MLflow/Prefect/Feast ~4 GB · LiteLLM + Langfuse ~6 GB → **≈ 31–33 GB**, leaving about 15 GB for builds, spikes and a small CPU model. Large models live in GPU memory on the Windows side (see 0.10).

Where: [WIN]. Create or replace `C:\Users\<you>\.wslconfig`:

#006 Windows PowerShell - copy & paste

```
@'
# C:\Users\<you>\.wslconfig — WSL2 VM settings for the "halo" lab (applies to ALL distros)
[wsl2]
memory=48GB                 # see sizing table; leaves >=48 GB for Windows + GPU
processors=16               # of 32 logical CPUs (Ryzen AI Max+ 395 = 16C/32T)
swap=16GB                   # protects against OOM-kills during image builds
networkingMode=nat          # recommended: single eth0, k3s node IP pinned to a dummy NIC
localhostForwarding=true    # Windows 127.0.0.1:<port> -> listening sockets in WSL (6443, 80, 443)
dnsTunneling=true           # DNS via Windows host; survives VPN/Wi-Fi changes
firewall=true               # Hyper-V firewall rules apply to the VM
vmIdleTimeout=-1            # never shut the VM down when idle (-1 = only on wsl --shutdown)
guiApplications=false       # no WSLg compositor: saves ~0.5 GB, not needed for a server

[experimental]
autoMemoryReclaim=gradual   # slowly hands unused page cache back to Windows
sparseVhd=true              # new ext4.vhdx files are sparse (shrink when you delete data)

[general]
instanceIdleTimeout=-1      # never stop the distro when no terminal is open (newer WSL; ignored if unknown)
'@ | Set-Content -Encoding ascii "$env:USERPROFILE\.wslconfig"
Get-Content "$env:USERPROFILE\.wslconfig"
```

> Changes take effect after `wsl --shutdown`. The **WSL Settings** app (Start → "WSL Settings") edits this same file, so use it if you want to look at the options in a GUI.

NAT vs mirrored: the short version

	NAT (recommended)	**Mirrored**
Linux interfaces	one `eth0`, 172.x, new IP each boot	copies of every Windows adapter (Wi-Fi, LAN, VPN)
Windows → cluster	`localhost` relay of *listening sockets*	shared stack, `localhost` both ways
LAN devices → cluster	needs `netsh portproxy`	works directly
What breaks with k3s	Only iptables-only ports (ServiceLB/hostPort) aren't relayed → solved by Traefik hostNetwork	k3s auto-picks the active Windows adapter → node IP changes when you switch networks or a VPN connects; IPv6 `::1` surprises; Windows apps can steal ports

Verify (after 0.4, once the distro is restarted): [WSL] `free -g` shows total ≈ 47 and `nproc` shows `16`.

If it fails 1. Settings ignored → the file has a `.txt` extension or the wrong encoding. Check with `Get-Item $env:USERPROFILE\.wslconfig`, then run `wsl --shutdown`. 2. `wsl` prints "Unknown key 'x'" → harmless for `[general]` / `[experimental]` keys on older builds. Update WSL.

0.4 `/etc/wsl.conf`: systemd, hostname, PATH hygiene (B1)

Goal: turn on systemd, set the hostname to `halo`, and keep Windows binaries off the Linux PATH. **Why:** `wsl.conf` configures *one distro*, roughly what cloud-init does for a VM. k3s installs as a systemd service, so systemd must be PID 1. `appendWindowsPath=false` stops `kubectl.exe` and `helm.exe` from Windows ever running inside Linux by accident. That's a classic "why is it talking to the wrong cluster" bug. **Where:** [WSL] (`wsl -d halo`)

#007 WSL Ubuntu shell (bash) - copy & paste

```
sudo tee /etc/wsl.conf >/dev/null <<'EOF'
# /etc/wsl.conf — per-distro settings for "halo"
[boot]
systemd=true                # k3s runs as a systemd unit

[network]
hostname=halo               # node name, kube context and overlay all match
generateHosts=true
generateResolvConf=true     # with dnsTunneling=true, DNS goes via Windows

[interop]
enabled=true                # can still call /mnt/c/Windows/System32/*.exe explicitly
appendWindowsPath=false     # keep kubectl.exe/helm.exe off the Linux PATH

[automount]
enabled=true
options="metadata,umask=22,fmask=11"

[user]
default=REPLACE_ME
EOF
sudo sed -i "s/REPLACE_ME/$USER/" /etc/wsl.conf
cat /etc/wsl.conf
```

Restart the distro from [WIN]:

#008 Windows PowerShell - copy & paste

```
wsl --shutdown
wsl -d halo
```

Back in [WSL]:

#009 WSL Ubuntu shell (bash) - copy & paste

```
hostname                      # halo
ps -p 1 -o comm=              # systemd
systemctl is-system-running   # running   (or "degraded" — see If it fails)
free -g | head -2; nproc      # ~47 total, 16
```

Basic packages, plus a `code` shortcut, since Windows' PATH isn't appended anymore:

#010 WSL Ubuntu shell (bash) - copy & paste

```
sudo apt-get update && sudo apt-get -y full-upgrade
sudo apt-get install -y curl git jq unzip bash-completion ca-certificates iptables conntrack
WINUSER=$(/mnt/c/Windows/System32/cmd.exe /c 'echo %USERNAME%' 2>/dev/null | tr -d '\r')
cat >> ~/.bashrc <<EOF

# --- halo lab ---
alias code='"/mnt/c/Users/${WINUSER}/AppData/Local/Programs/Microsoft VS Code/bin/code"'
alias explorer='/mnt/c/Windows/explorer.exe'
EOF
source ~/.bashrc
```

- ☐ `hostname` = halo, PID 1 = systemd, `nproc` = 16

If it fails 1. `systemctl is-system-running` → `degraded`: list failures with `systemctl --failed`. On WSL, `systemd-remount-fs` or `multipathd` failures are common and harmless. Mask them with `sudo systemctl mask multipathd.service multipathd.socket` if they show up. 2. Hostname is still random → the `[network]` section has a typo, or you didn't run `wsl --shutdown` (closing the window isn't enough). 3. The `code` alias fails → VS Code was installed system-wide. Use `"/mnt/c/Program Files/Microsoft VS Code/bin/code"`.

0.5 Kernel and networking prerequisites for k3s (B2)

Goal: prove the WSL kernel has everything k3s and flannel need, and make the settings persistent. **Why:** Kubernetes networking is netfilter rules plus a bridge plus an overlay. `br_netfilter` lets iptables see bridged pod traffic, `overlay` is containerd's image filesystem, cgroup v2 is

how the kubelet enforces CPU and memory limits, and shared mount propagation lets volumes mount into pods. The WSL 6.18 kernel has all of these, but some are **modules** (`=m`) that need loading at boot. In Ansible terms, this is your `prereqs` role. **Where:** [WSL]

Make it persistent first:

#011 WSL Ubuntu shell (bash) - copy & paste

```
sudo tee /etc/modules-load.d/k3s.conf >/dev/null <<'EOF'
# /etc/modules-load.d/k3s.conf — loaded at every boot by systemd-modules-load
br_netfilter
overlay
dummy
EOF

sudo tee /etc/sysctl.d/90-k3s.conf >/dev/null <<'EOF'
# /etc/sysctl.d/90-k3s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
# Argo CD, Prometheus, Loki and friends watch many files; WSL defaults (128) are too low
fs.inotify.max_user_instances       = 8192
fs.inotify.max_user_watches         = 524288
vm.max_map_count                    = 262144
EOF

sudo systemctl restart systemd-modules-load
sudo sysctl --system >/dev/null
```

Then run the checks. Each line prints `OK`, or `FIX` plus what to look at:

#012 WSL Ubuntu shell (bash) - copy & paste

```
chk(){ if eval "$2" >/dev/null 2>&1; then echo "OK   $1"; else echo "FIX  $1  -> $3"; fi; }
chk "systemd is PID 1"            '[ "$(ps -p1 -o comm=)" = systemd ]'                        "wsl.conf [boot] systemd=true + wsl --shutdown"
chk "cgroup v2 unified"           '[ "$(stat -fc %T /sys/fs/cgroup)" = cgroup2fs ]'            "appendix T2 (kernelCommandLine)"
chk "cgroup controllers"          '( for c in memory cpuset pids; do grep -qw $c /sys/fs/cgroup/cgroup.controllers || exit 1; done )' "wsl --update"
chk "kernel modules available"    '[ -d /lib/modules/$(uname -r) ]'                            "wsl --update (modules ship with WSL)"
chk "br_netfilter loaded"         'lsmod | grep -qw br_netfilter'                              "sudo modprobe br_netfilter"
chk "overlayfs"                   'grep -qw overlay /proc/filesystems'                         "sudo modprobe overlay"
chk "dummy NIC module"            'sudo modprobe dummy'                                        "wsl --update"
chk "bridge-nf-call-iptables=1"   '[ "$(sysctl -n net.bridge.bridge-nf-call-iptables)" = 1 ]'  "sudo sysctl --system"
chk "ip_forward=1"                '[ "$(sysctl -n net.ipv4.ip_forward)" = 1 ]'                 "sudo sysctl --system"
chk "iptables works (nf_tables)"  'sudo iptables -t nat -L -n'                                 "sudo update-alternatives --set iptables /usr/sbin/iptables-nft"
chk "root mount is shared"        'findmnt -no PROPAGATION / | grep -q shared'                  "sudo mount --make-rshared /  (systemd normally does this)"
chk "/dev/kmsg exists"            '[ -e /dev/kmsg ]'                                           "systemd off? see T1"
chk "clock in sync"               'timedatectl show -p NTPSynchronized --value | grep -q yes || true' "sudo hwclock -s"
iptables --version
```

Expected: every line starts with `OK`, and `iptables v1.8.10 (nf_tables)`.

iptables vs nftables, in plain words: Ubuntu 24.04's `iptables` command is a front end that writes **nftables** rules (`iptables-nft`). kube-proxy and flannel still speak iptables. Problems only start if two different iptables front ends write to the same kernel. In Step 1, k3s runs with `prefer-bundled-bin: true` so it uses its own tested iptables binaries every time.

- ☐ All checks `OK`

If it fails 1. `kernel modules available` = FIX → WSL is old or the modules VHD didn't mount. Run `wsl --update`, then `wsl --shutdown`. 2. `cgroup v2 unified` = FIX → appendix **T2** (add `kernelCommandLine = cgroup_no_v1=all systemd.unified_cgroup_hierarchy=1` under `[wsl2]`). 3. `iptables works` fails with "Could not fetch rule set generation id" → `sudo update-alternatives --set iptables /usr/sbin/iptables-nft` (k3s's bundled binaries avoid this anyway).

0.6 Keep it running: idle timeouts, logon task, no sleep (B2)

Goal: halo is up after every logon and stays up with no terminal open. **Why:** by default WSL stops a distro about 15 s after the last terminal closes, and the VM about 60 s after that. That's fine for dev tools, but fatal for a cluster hosting Phoenix. There are three layers: the idle timeouts set to `-1` (0.3), a scheduled task that boots the distro at logon and holds one process open (belt and braces), and Windows never sleeping on AC. It's the WSL version of setting a VM to "start automatically with the host". **Where:** [WIN-ADMIN]

#013 Windows PowerShell - copy & paste

```
$action   = New-ScheduledTaskAction -Execute "$env:WINDIR\System32\conhost.exe" `
            -Argument '--headless wsl.exe -d halo --exec /bin/sh -c "exec sleep infinity"'
$trigger  = New-ScheduledTaskTrigger -AtLogOn -User "$env:USERDOMAIN\$env:USERNAME"
$trigger.Delay = 'PT20S'
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
            -ExecutionTimeLimit ([TimeSpan]::Zero) -MultipleInstances IgnoreNew `
            -RestartCount 3 -RestartInterval (New-TimeSpan -Minutes 1)
Register-ScheduledTask -TaskName 'WSL-halo-keepalive' -Action $action -Trigger $trigger -Settings $settings `
  -Description 'Boots WSL distro halo (k3s) at logon and keeps it alive' -Force

# Never sleep or hibernate on AC power; the screen may still turn off
powercfg /change standby-timeout-ac 0
powercfg /change hibernate-timeout-ac 0
powercfg /change monitor-timeout-ac 15

Start-ScheduledTask -TaskName 'WSL-halo-keepalive'
Start-Sleep 5; wsl -l -v
```

Expected: `halo Running 2`. Close every terminal, wait 2 minutes, run `wsl -l -v` again, and it should still say **Running**.

> **Unattended reboots (Windows Update at 3 a.m.):** an *AtLogOn* task only fires when you log in. For real around-the-clock uptime later, add an *AtStartup* trigger and register the task with `-User $env:USERNAME -Password (Read-Host)` ("run whether user is logged on or not"), or set *Active hours* in Windows Update so reboots happen while you're around. Note it now; decide before Phoenix goes live.

- ☐ Task registered; distro still Running 2 minutes after closing all terminals
- ☐ Sleep on AC = Never

If it fails 1. The task shows "Last Run Result 0x1" → `wsl.exe` ran before WSL was ready. The 20 s delay normally covers it, so increase it to `PT60S`. 2. The distro still stops → check `wsl --version` (3.0.x) and that `.wslconfig` has `vmIdleTimeout=-1`, then run `wsl --shutdown` and start the task again. 3. A console window appears → `conhost --headless` isn't supported on your build. Use `-Execute wsl.exe -Argument '-d halo --exec /bin/sh -c "exec sleep infinity"'` and accept a minimised window.

0.7 Filesystem layout and disk hygiene (B2)

Goal: know where data lives, and keep it on fast ext4. **Why:** the distro's root filesystem is an `ext4.vhdx` (a virtual disk, like a thin-provisioned VMDK). `/mnt/c` is a 9P network share into Windows, often **10–50× slower** for many small files, with no proper Linux permissions. Git repos,

container images, etcd and PVs all belong on `~/` or `/var/lib`, never on `/mnt/c`.

What	Path (inside halo)	On Windows
Your repo	`~/src/ai-platform-portfolio`	`\\wsl.localhost\halo\home\<you>\src\…` (open with VS Code WSL, not Explorer)
k3s data (etcd, certs, images)	`/var/lib/rancher/k3s`	inside `ext4.vhdx`
local-path PVs	`/var/lib/rancher/k3s/storage/pvc-<uid>_<ns>_<pvc>`	inside `ext4.vhdx`
The disk file	—	`%LOCALAPPDATA%\wsl\{guid}\ext4.vhdx` (or your `--location`)

Find the vhdx and its size ([WIN]):

#014 Windows PowerShell - copy & paste

```
Get-ChildItem HKCU:\Software\Microsoft\Windows\CurrentVersion\Lxss | ForEach-Object {
  $p = Get-ItemProperty $_.PSPath; if ($p.DistributionName -eq 'halo') {
    $v = Join-Path $p.BasePath 'ext4.vhdx'; "{0}  {1:N1} GB" -f $v, ((Get-Item $v).Length/1GB) } }
```

How it grows and shrinks: the vhdx grows on demand (up to 1 TB by default) and does **not** shrink on its own when you delete files. With `sparseVhd=true` (set before install) it gives space back after a trim. Monthly, or after deleting big images or models:

#015 WSL Ubuntu shell (bash) - copy & paste

```
# [WSL] tell the virtual disk which blocks are free
sudo fstrim -av
```

#016 Windows PowerShell - copy & paste

```
# [WIN-ADMIN] if the file is still big: shut down, then compact
wsl --shutdown
wsl --manage halo --set-sparse true        # no-op if already sparse
# Manual compaction (works on Home and Pro):
#   diskpart → select vdisk file="<path from above>" → attach vdisk readonly → compact vdisk → detach vdisk → exit
```

- ☐ I know where my vhdx lives and how big it is

0.8 GPU flag (information only, don't set up now)

Question: will WSL block L2 (llama.cpp on the Radeon 8060S) or the bigger goal of several self-hosted models?

Status (Nov 2026): - **ROCm on WSL:** since **ROCm 7.2.1 + Adrenalin 26.2.2**, AMD officially supports **Strix Halo (gfx1151)** under WSL2, through the open-source **ROCDXG** (`librocdxg`) path over `/dev/dxg`. Known gaps remain (for example, `hipMallocManaged` isn't supported on gfx1151), and you need WSL-specific runtime packages, not the standard `hsa-rocr`. Old guides that use `HSA_OVERRIDE_GFX_VERSION` / gfx1100 workarounds are out of date. - **Vulkan in WSL:** only through Mesa's *Dozen* (Vulkan-on-D3D12). It works but is slow. Not recommended for llama.cpp. - **DirectML:** Windows-native ML API. Not a fit for llama.cpp. - **Inside Kubernetes:** the AMD GPU device plugin expects `/dev/kfd` + `/dev/dri`, and WSL exposes `/dev/dxg`. A pod *can* use the GPU through hostPath mounts of `/dev/dxg` and `/usr/lib/wsl`, but with **no device-plugin scheduling or accounting**. That isn't how production GPU nodes work.

Verdict: WSL does **not** block L2. Recommended path for L2: run **llama.cpp natively on Windows** (Vulkan or HIP backend; mature and fast on Strix Halo, with full access to the GPU memory share), and register it inside k3s as an **external backend**: a Service without a selector plus an EndpointSlice pointing at the Windows host IP (give the PC a DHCP reservation). LiteLLM in k3s routes to it, applies budgets and falls back. Stretch goal: a llama.cpp pod using ROCDXG via `/dev/dxg`, to mimic a "GPU node pool". Real Kubernetes GPU scheduling (device plugin / GPU Operator, MIG, bin-packing) is what the **AWS** labs are for.

Impact on today's choices: keep WSL at ≤ 48 GB so the GPU share can grow, keep model files on a Windows NTFS drive (`D:\models`), and keep k3s's node IP stable so the external-backend pattern stays simple.

0.9 First baseline snapshot (B2)

Goal: a whole-distro backup of the clean, prepared OS before k3s. You restore it in the Step 9 rebuild drill. **Why:** `wsl --export` writes the distro's root filesystem to a tarball, like a VM template or a cold snapshot in vSphere. It's your fastest "back to known-good" button. Terminate the distro first so nothing is writing. **Where:** [WIN]

#017 Windows PowerShell - copy & paste

```
$bk = "$env:USERPROFILE\WSL-backups"; New-Item -ItemType Directory -Force $bk | Out-Null
wsl --terminate halo
wsl --export halo "$bk\halo-baseline-$(Get-Date -Format yyyyMMdd).tar"
Get-ChildItem $bk | Select-Object Name,@{n='GB';e={[math]::Round($_.Length/1GB,2)}}
Start-ScheduledTask -TaskName 'WSL-halo-keepalive'
```

Expected: `Export in progress, this may take a few minutes. The operation completed successfully.` The file is about 1.5–3 GB.

> **Treat every export as a secret.** From Step 1 on, exports contain the cluster CA, tokens and Argo CD credentials. Keep them out of OneDrive, Git, or anything you'd make public. Put a copy on an external disk.

- ☐ `halo-baseline-YYYYMMDD.tar` exists

If it fails: "The file is in use" or the export hangs → run `wsl --shutdown` and try again. Not enough space → export straight to another drive (`D:\WSL-backups`).

CKA corner: Step 0

- **Domain:** Cluster Architecture, Installation & Configuration (25%): node prerequisites for kubeadm are the same (modules, sysctls, cgroups, swap, container runtime).
- **Practise:** `lsmod | grep br_netfilter` · `sysctl net.ipv4.ip_forward` · `stat -fc %T /sys/fs/cgroup` · `systemctl status <unit>` · `journalctl -u <unit> -f`
- **Drill:** write `/etc/modules-load.d/k8s.conf` and `/etc/sysctl.d/k8s.conf` from memory in under 2 minutes (the kubeadm "install container runtime" page asks for exactly this).

Interview angle: "I treat even a laptop lab like a fleet node: pinned OS image, hardware sizing from a capacity budget, kernel prerequisites written as code, and a golden snapshot before the platform layer." **Phoenix angle:** the keep-alive task, no-sleep policy and memory budget are what let Phoenix run around the clock on a PC you also use every day.

Block B2 done when: ☐ baseline export exists · ☐ distro survives closing all terminals.

Steps 1–3: k3s, client tools, cluster verification (blocks B3–B4)

Native k3s vs k3d / Rancher Desktop: k3d runs k3s *inside Docker containers*. That's great for throwaway multi-node CI clusters, but it adds Docker as a second runtime and a second network layer, and your long-lived Phoenix workloads would then depend on both. Rancher Desktop wraps k3s in its own VM and GUI, and hides the systemd unit, config file, certs and etcd. Those are exactly what the CKA tests and what you'd run in production. **Native k3s as a systemd service is the right choice here.** Use k3d later only for disposable test clusters, such as CI eval gates.

Step 1: Install k3s (single node, pinned) (B3)

1.1 Stable node IP: dummy interface `halo0`

Goal: a network interface whose IP (`10.255.255.10`) never changes, for k3s, etcd and flannel to bind to. **Why:** in NAT mode, `eth0` gets a new 172.x address on every WSL boot. Embedded etcd records its peer URL by IP, and the API server certificate lists IPs, so a moving IP is the top cause of "my WSL cluster died after a reboot". A *dummy* interface is a virtual NIC that's always up, like a loopback adapter on a Windows server or a VMkernel port with a fixed IP. Outbound traffic still leaves through `eth0`. **Where:** [WSL]

#018 WSL Ubuntu shell (bash) - copy & paste

```
sudo tee /etc/systemd/system/halo-node-ip.service >/dev/null <<'EOF'
# /etc/systemd/system/halo-node-ip.service
[Unit]
Description=Stable k3s node IP on dummy interface halo0 (10.255.255.10)
DefaultDependencies=no
After=systemd-modules-load.service
Before=network-pre.target k3s.service
Wants=network-pre.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/bin/sh -c 'ip link show halo0 >/dev/null 2>&1 || ip link add halo0 type dummy'
ExecStart=/usr/sbin/ip addr replace 10.255.255.10/32 dev halo0
ExecStart=/usr/sbin/ip link set halo0 up
ExecStop=/usr/sbin/ip link del halo0
```

```
[Install]
WantedBy=multi-user.target
EOF

sudo mkdir -p /etc/systemd/system/k3s.service.d
sudo tee /etc/systemd/system/k3s.service.d/10-halo-node-ip.conf >/dev/null <<'EOF'
# /etc/systemd/system/k3s.service.d/10-halo-node-ip.conf — k3s must start after halo0 exists
[Unit]
Requires=halo-node-ip.service
After=halo-node-ip.service
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now halo-node-ip.service
ip -br addr show halo0
```

Expected: `halo0 UNKNOWN 10.255.255.10/32` (`UNKNOWN` is normal for a dummy interface).

> **Address choice:** `10.255.255.0/24` stays clear of the k3s pod CIDR (`10.42.0.0/16`), the service CIDR (`10.43.0.0/16`) and the WSL NAT range (172.16–31.x). If a VPN you use later claims `10.255.x`, change it in three files: this unit, the k3s config, and the Traefik config.

1.2 k3s configuration file

Goal: all k3s settings in one declarative file. No long command-line flags. **Why:** `/etc/rancher/k3s/config.yaml` mirrors the `k3s server` flags one to one. It's the equivalent of a kubeadm `ClusterConfiguration`, or an Ansible vars file. Each setting below has a comment saying *why*. In Step 4 this file goes into Git (it holds no secrets). **Where:** [WSL]

#019 WSL Ubuntu shell (bash) - copy & paste

```
sudo mkdir -p /etc/rancher/k3s
sudo tee /etc/rancher/k3s/config.yaml >/dev/null <<'EOF'
# /etc/rancher/k3s/config.yaml — k3s server config for node "halo"
# Source of truth: platform/k3s/halo/config.yaml in ai-platform-portfolio. No secrets in this file.

# --- identity & addressing -------------------------------------------------
node-name: halo
node-ip: 10.255.255.10            # dummy NIC halo0: survives WSL IP changes
advertise-address: 10.255.255.10  # IP the apiserver advertises to the cluster
flannel-iface: halo0              # pin flannel (pod network) to the stable NIC
```

```
cluster-cidr: 10.42.0.0/16        # k3s default, written down on purpose
service-cidr: 10.43.0.0/16        # k3s default, written down on purpose

# --- API server certificate SANs (what clients may call it) ----------------
tls-san:
  - halo
  - halo.localhost
  - localhost
  - 127.0.0.1                     # Windows kubeconfig uses this via localhost forwarding
  - 10.255.255.10

# --- datastore: embedded etcd (production-like; CKA snapshot practice) -----
cluster-init: true
etcd-snapshot-schedule-cron: "0 */6 * * *"   # every 6h
etcd-snapshot-retention: 12                  # keep 3 days

# --- security --------------------------------------------------------------
secrets-encryption: true          # encrypt Secret objects at rest in etcd
write-kubeconfig-mode: "0600"     # admin kubeconfig readable by root only

# --- runtime ---------------------------------------------------------------
prefer-bundled-bin: true          # use k3s's tested iptables/ipset, not the host's

# --- node labels & kubelet capacity protection -----------------------------
node-label:
  - "lab.platform/env=halo"
kubelet-arg:
  - "system-reserved=cpu=500m,memory=1Gi"      # keep room for systemd, WSL init, sshd...
  - "kube-reserved=cpu=500m,memory=1Gi"        # ...and for k3s itself
  - "eviction-hard=memory.available<500Mi,nodefs.available<10%,imagefs.available<10%"
  - "image-gc-high-threshold=80"
  - "image-gc-low-threshold=70"

# --- bundled add-ons: all KEPT (nothing under 'disable:') ------------------
# traefik       -> ingress controller (configured by traefik-config.yaml below)
# servicelb     -> LoadBalancer services via host ports (klipper-lb)
# local-storage -> local-path StorageClass for PVCs
```

```
# metrics-server, coredns
EOF
```

1.3 Traefik override: bind 80/443 for real (HelmChartConfig)

Goal: Traefik listens on real sockets on ports 80/443 so Windows `localhost` reaches it. **Why:** by default k3s exposes Traefik through **ServiceLB**, which uses *iptables* rules to grab host ports. WSL's localhost relay only forwards ports that have a real *listening socket*, so those iptables-only ports would be invisible from Windows. Running Traefik with `hostNetwork: true` is the standard bare-metal "edge node" pattern: the ingress controller binds the node's ports directly. k3s lets you customise its bundled charts with a `HelmChartConfig` dropped into its auto-deploy folder. **Where:** [WSL]

#020 WSL Ubuntu shell (bash) - copy & paste

```
sudo mkdir -p /var/lib/rancher/k3s/server/manifests
sudo tee /var/lib/rancher/k3s/server/manifests/traefik-config.yaml >/dev/null <<'EOF'
# /var/lib/rancher/k3s/server/manifests/traefik-config.yaml
# Source of truth: platform/k3s/halo/traefik-config.yaml
apiVersion: helm.cattle.io/v1
kind: HelmChartConfig
metadata:
  name: traefik
  namespace: kube-system
spec:
  valuesContent: |-
    hostNetwork: true                # real sockets on :80/:443 -> WSL localhost relay works
    service:
      type: ClusterIP                # no ServiceLB pod fighting for the same host ports
    ports:
      web:
        port: 80
      websecure:
        port: 443
      traefik:
        port: 9000                   # internal ping/dashboard entrypoint, off :8080
      metrics:
        port: 9102                   # keep :9100 free for node-exporter later
    updateStrategy:                  # hostNetwork pods can't surge on the same ports
      type: RollingUpdate
      rollingUpdate:
```

```
      maxUnavailable: 1
      maxSurge: 0
  securityContext:                  # binding <1024 needs NET_BIND_SERVICE as root
    capabilities:
      drop: [ALL]
      add: [NET_BIND_SERVICE]
    readOnlyRootFilesystem: true
    runAsNonRoot: false
    runAsUser: 0
    runAsGroup: 0
  providers:
    kubernetesIngress:
      publishedService:
        enabled: false
  additionalArguments:
    # Write this IP into every Ingress status -> Argo CD sees Ingresses as Healthy
    - "--providers.kubernetesingress.ingressendpoint.ip=10.255.255.10"
EOF
```

1.4 Install k3s (pinned script + pinned version)

Goal: k3s v1.36.5+k3s1 running as the `k3s` systemd service. **Why:** piping `get.k3s.io` straight into `sh` runs an unpinned script from the internet. Download the installer **from the release tag**, read it, then run it with `INSTALL_K3S_VERSION`. The installer checks the binary's SHA-256, writes `k3s.service`, and creates the `kubectl`, `crictl` and `ctr` symlinks plus `k3s-uninstall.sh`. In production this would be an Ansible role or a golden image. **Where:** [WSL]

#021 WSL Ubuntu shell (bash) - copy & paste

```
K3S_VERSION='v1.36.5+k3s1'
curl -fsSL -o /tmp/k3s-install.sh "https://raw.githubusercontent.com/k3s-io/k3s/${K3S_VERSION/+/%2B}/install.sh"
head -5 /tmp/k3s-install.sh            # skim it; it's a long but readable shell script
sudo env INSTALL_K3S_VERSION="$K3S_VERSION" sh /tmp/k3s-install.sh
```

Expected (last lines):

Output / reference - not a command

```
[INFO]  Using v1.36.5+k3s1 as release
[INFO]  Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.36.5+k3s1/sha256sum-amd64.txt
[INFO]  Verifying binary download
[INFO]  Creating /usr/local/bin/kubectl symlink to k3s
[INFO]  Creating /usr/local/bin/crictl symlink to k3s
[INFO]  systemd: Enabling k3s unit
[INFO]  systemd: Starting k3s
```

Set up your own kubeconfig. The context, cluster and user are all named `halo`:

#022 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p ~/.kube
sudo cat /etc/rancher/k3s/k3s.yaml | sed -e 's/: default$/: halo/' > ~/.kube/config
chmod 600 ~/.kube/config
kubectl config get-contexts
kubectl get nodes -o wide
```

Expected:

Output / reference - not a command

```
CURRENT   NAME   CLUSTER   AUTHINFO   NAMESPACE
*         halo   halo      halo
NAME   STATUS   ROLES                AGE   VERSION        INTERNAL-IP     ...  CONTAINER-RUNTIME
halo   Ready    control-plane,etcd   60s   v1.36.5+k3s1   10.255.255.10   ...  containerd://2.x
```

Verify:

#023 WSL Ubuntu shell (bash) - copy & paste

```
systemctl is-active k3s halo-node-ip                # active, active
sudo k3s check-config | tail -5                     # STATUS: pass
sudo k3s secrets-encrypt status | head -3           # Encryption Status: Enabled
kubectl -n kube-system get helmchart,pods           # helm-install-traefik* Completed, traefik Running
kubectl -n kube-system get pods -l app.kubernetes.io/name=traefik -o wide   # IP = 10.255.255.10 (hostNetwork)
sudo ss -ltnp | grep -E ':(80|443|6443)\s'          # traefik on :80/:443, k3s on :6443
```

- ☐ `halo` Ready with INTERNAL-IP 10.255.255.10
- ☐ Traefik listening on :80 and :443; secrets encryption enabled

If it fails 1. k3s keeps restarting → `sudo journalctl -u k3s -e --no-pager | tail -50`. Look for `iptables` (→ appendix **T3**), `cgroup` (→ **T2**), or `failed to find interface halo0` (→ check that `halo-node-ip.service` is active). 2. Traefik pod `Pending` / `CrashLoopBackOff` → `kubectl -n kube-system logs job/helm-install-traefik` shows chart value errors. `bind: permission denied` means the securityContext block wasn't applied. Edit the file, and k3s re-applies it within seconds. 3. Something on Windows already holds port 80, so the relay can't forward → appendix **T7**.

What the bundled add-ons do (kept on purpose)

Add-on	What it is	Why keep it	Later
Traefik	Ingress controller: routes `Host:` / path rules to Services	Lets every app get a name like `x.halo.localhost`	Gateway API routes, TLS with cert-manager
ServiceLB (klipper-lb)	Implements `type: LoadBalancer` by grabbing host ports with iptables	Ready for TCP services (for example Postgres for a desktop client) without MetalLB	Not used by Traefik here (hostNetwork); still available
local-path	Dynamic PV provisioner: each PVC becomes a folder on the node	Zero-setup storage for Timescale/Redis	Phoenix needs a **Retain** StorageClass + backups (see Phoenix angle)
metrics-server	Resource metrics API	`kubectl top`, HPA	Prometheus later
CoreDNS	Cluster DNS	Service discovery	—

k3s vs kubeadm: what k3s hides that the CKA expects you to know

Topic	k3s (halo)	kubeadm (CKA exam)
Control plane	**One process** `k3s server` runs apiserver, scheduler, controller-manager, kubelet, kube-proxy and containerd. `kubectl -n kube-system get pods` shows **no** apiserver pods	Static pods in `/etc/kubernetes/manifests/` (`kube-apiserver.yaml`, `etcd.yaml`, ...), run by the kubelet
Datastore	SQLite by default (via *kine*); here **embedded etcd** in the k3s process, data in `/var/lib/rancher/k3s/server/db/etcd`	etcd static pod, data in `/var/lib/etcd`; backup with `etcdctl snapshot save`
Certificates	`/var/lib/rancher/k3s/server/tls/`	`/etc/kubernetes/pki/` (`kubeadm certs check-expiration`)
Admin kubeconfig	`/etc/rancher/k3s/k3s.yaml`	`/etc/kubernetes/admin.conf`
Kubelet config	flags from `config.yaml` (`kubelet-arg`)	`/var/lib/kubelet/config.yaml` + `kubelet.service` drop-in
Container runtime	bundled containerd, socket `/run/k3s/containerd/containerd.sock`; `sudo k3s crictl ps`	system containerd, `/run/containerd/containerd.sock`; `crictl ps`
CNI	flannel (VXLAN) built in; config under `/var/lib/rancher/k3s/agent/etc/cni/net.d`	you install one (Calico/Flannel/Cilium); `/etc/cni/net.d`
Auto-applied manifests	`/var/lib/rancher/k3s/server/manifests/` (k3s applies them like a mini-GitOps)	none (static pods are a different mechanism)

Topic	k3s (halo)	kubeadm (CKA exam)
Static pods	`/var/lib/rancher/k3s/agent/pod-manifests/`	`/etc/kubernetes/manifests/`
Upgrades	swap the binary (`INSTALL_K3S_VERSION=…`) or the system-upgrade-controller	`kubeadm upgrade plan/apply` , drain, upgrade kubelet per node

Exam trap: on a kubeadm cluster, a broken control plane is fixed by editing the static pod YAML. On k3s there's no YAML, only `config.yaml` plus `systemctl restart k3s` . Practise both.

CKA corner: Step 1

- **Domain:** Cluster Architecture, Installation & Configuration (25%).
- **Practise:** `kubectl get nodes -o wide` · `kubectl describe node halo | grep -A6 Allocatable` · `kubectl get --raw='/readyz?verbose'` · `sudo k3s crictl ps` · `sudo k3s etcd-snapshot save --name manual-1 && sudo k3s etcd-snapshot ls`
- **Drill:** `kubectl label node halo disk=ssd` → `kubectl get nodes -L disk` → `kubectl label node halo disk-` (labels and selectors come up a lot).

Interview angle: "I pinned the node identity to a stable interface so etcd and certificates survive network changes, ran embedded etcd with scheduled snapshots, encrypted Secrets at rest, and reserved kubelet capacity: the same hardening I'd ask for on a production node pool."
Phoenix angle: etcd snapshots every 6 h plus `secrets-encryption` mean Phoenix's cluster state is recoverable and its future Secrets aren't stored in plain text.

Step 2: Client tools in WSL and on Windows (B4)

Goal: pinned `kubectl` , `helm` and `argocd` (plus optional `k9s`) in both shells. Windows uses the `halo` context. **Why:** WSL is where you'll *operate* the platform (scripts, Git). Windows is where your browser, VS Code and habits live. Same versions on both sides means no "works in one shell" surprises. Every download is checksum-verified, as you'd do in a Jenkins agent image.

2.1 In WSL ([WSL])

#024 WSL Ubuntu shell (bash) - copy & paste

```
KV=v1.36.6; HV=v4.3.1; AV=v3.5.6; K9V=v0.51.0
T=$(mktemp -d); cd "$T"

# kubectl (replace k3s's symlink with the pinned standalone binary)
curl -fsSLO "https://dl.k8s.io/release/${KV}/bin/linux/amd64/kubectl"
curl -fsSLO "https://dl.k8s.io/release/${KV}/bin/linux/amd64/kubectl.sha256"
echo "$(cat kubectl.sha256)  kubectl" | sha256sum --check
sudo rm -f /usr/local/bin/kubectl            # removes the SYMLINK only; /usr/local/bin/k3s stays
```

```
sudo install -m 0755 kubectl /usr/local/bin/kubectl

# helm
curl -fsSLO "https://get.helm.sh/helm-${HV}-linux-amd64.tar.gz"
curl -fsSLO "https://get.helm.sh/helm-${HV}-linux-amd64.tar.gz.sha256sum"
sha256sum --check "helm-${HV}-linux-amd64.tar.gz.sha256sum"
tar xzf "helm-${HV}-linux-amd64.tar.gz" && sudo install -m 0755 linux-amd64/helm /usr/local/bin/helm

# argocd CLI
curl -fsSLO "https://github.com/argoproj/argo-cd/releases/download/${AV}/argocd-linux-amd64"
curl -fsSLO "https://github.com/argoproj/argo-cd/releases/download/${AV}/cli_checksums.txt"
grep ' argocd-linux-amd64$' cli_checksums.txt | sha256sum --check
sudo install -m 0755 argocd-linux-amd64 /usr/local/bin/argocd

# k9s (optional)
curl -fsSLO "https://github.com/derailed/k9s/releases/download/${K9V}/k9s_Linux_amd64.tar.gz"
curl -fsSLO "https://github.com/derailed/k9s/releases/download/${K9V}/checksums.sha256"
grep ' k9s_Linux_amd64.tar.gz$' checksums.sha256 | sha256sum --check
tar xzf k9s_Linux_amd64.tar.gz k9s && sudo install -m 0755 k9s /usr/local/bin/k9s
cd ~ && rm -rf "$T"

cat >> ~/.bashrc <<'EOF'
# --- k8s tooling (CKA habits) ---
source <(kubectl completion bash)
source <(helm completion bash)
source <(argocd completion bash)
alias k=kubectl
complete -o default -F __start_kubectl k
export do="--dry-run=client -o yaml"      # k create deploy x --image=y $do > x.yaml
export now="--force --grace-period 0"     # k delete pod x $now
# argocd CLI reaches the API through a kubectl port-forward (no DNS/TLS needed)
export ARGOCD_OPTS="--port-forward --port-forward-namespace argocd --plaintext"
EOF
source ~/.bashrc
kubectl version; helm version --short; argocd version --client --short
```

Expected: `Client Version: v1.36.6` · `Server Version: v1.36.5+k3s1` · `v4.3.1+g…` · `argocd: v3.5.6+…`

Copy the kubeconfig to Windows and lock down its permissions:

#025 WSL Ubuntu shell (bash) - copy & paste

```
WINHOME=$(wslpath "$(/mnt/c/Windows/System32/cmd.exe /c 'echo %USERPROFILE%' 2>/dev/null | tr -d '\r')")
mkdir -p "$WINHOME/.kube"
[ -f "$WINHOME/.kube/config" ] && cp "$WINHOME/.kube/config" "$WINHOME/.kube/config.bak-$(date +%s)"
cp ~/.kube/config "$WINHOME/.kube/config"
```

2.2 On Windows ([WIN])

#026 Windows PowerShell - copy & paste

```
$bin = "$env:USERPROFILE\tools\bin"; New-Item -ItemType Directory -Force $bin | Out-Null
$KV='v1.36.6'; $HV='v4.3.1'; $AV='v3.5.6'; $K9V='v0.51.0'
$ProgressPreference = 'SilentlyContinue'; $tmp = New-Item -ItemType Directory -Force "$env:TEMP\halo-tools"
function Assert-Hash($file, $expected) {
  if ((Get-FileHash $file -Algorithm SHA256).Hash -ine $expected.Trim()) { throw "Checksum mismatch: $file" } else { "OK  $file" }
}

# kubectl
Invoke-WebRequest "https://dl.k8s.io/release/$KV/bin/windows/amd64/kubectl.exe" -OutFile "$bin\kubectl.exe"
Invoke-WebRequest "https://dl.k8s.io/release/$KV/bin/windows/amd64/kubectl.exe.sha256" -OutFile "$tmp\kubectl.sha256"
Assert-Hash "$bin\kubectl.exe" (Get-Content "$tmp\kubectl.sha256" -Raw)

# helm
Invoke-WebRequest "https://get.helm.sh/helm-$HV-windows-amd64.zip" -OutFile "$tmp\helm.zip"
Invoke-WebRequest "https://get.helm.sh/helm-$HV-windows-amd64.zip.sha256sum" -OutFile "$tmp\helm.sha256sum"
Assert-Hash "$tmp\helm.zip" ((Get-Content "$tmp\helm.sha256sum" -Raw).Split(' ')[0])
Expand-Archive "$tmp\helm.zip" -DestinationPath $tmp -Force; Copy-Item "$tmp\windows-amd64\helm.exe" $bin -Force

# argocd
Invoke-WebRequest "https://github.com/argoproj/argo-cd/releases/download/$AV/argocd-windows-amd64.exe" -OutFile "$bin\argocd.exe"
Invoke-WebRequest "https://github.com/argoproj/argo-cd/releases/download/$AV/cli_checksums.txt" -OutFile "$tmp\argocd.sums"
Assert-Hash "$bin\argocd.exe" ((Select-String -Path "$tmp\argocd.sums" -Pattern 'argocd-windows-amd64.exe$').Line.Split(' ')[0])

# k9s (optional)
Invoke-WebRequest "https://github.com/derailed/k9s/releases/download/$K9V/k9s_Windows_amd64.zip" -OutFile "$tmp\k9s.zip"
Invoke-WebRequest "https://github.com/derailed/k9s/releases/download/$K9V/checksums.sha256" -OutFile "$tmp\k9s.sums"
Assert-Hash "$tmp\k9s.zip" ((Select-String -Path "$tmp\k9s.sums" -Pattern 'k9s_Windows_amd64.zip$').Line.Split(' ')[0])
```

```
Expand-Archive "$tmp\k9s.zip" -DestinationPath "$tmp\k9s" -Force; Copy-Item "$tmp\k9s\k9s.exe" $bin -Force

# PATH + argocd options (user scope), then lock down the kubeconfig (cluster-admin credentials!)
$userPath = [Environment]::GetEnvironmentVariable('Path','User')
if ($userPath -notlike "*$bin*") { [Environment]::SetEnvironmentVariable('Path', "$bin;$userPath", 'User') }
[Environment]::SetEnvironmentVariable('ARGOCD_OPTS','--port-forward --port-forward-namespace argocd --plaintext','User')
icacls "$env:USERPROFILE\.kube\config" /inheritance:r /grant:r "$($env:USERNAME):(R,W)" | Out-Null
```

Open a **new** PowerShell window (so the PATH change applies), then:

#027 Windows PowerShell - copy & paste

```
kubectl config get-contexts
kubectl --context halo get nodes
helm version --short; argocd version --client --short
```

Expected: context `halo` marked current; `halo Ready control-plane,etcd … v1.36.5+k3s1`.

- ☐ WSL: kubectl/helm/argocd at pinned versions, `k` alias works
- ☐ Windows: same versions; `kubectl get nodes` works through `https://127.0.0.1:6443`

If it fails 1. Windows `Unable to connect … 127.0.0.1:6443` → the distro isn't running (`wsl -l -v`), or `localhostForwarding` is off. Run `Test-NetConnection 127.0.0.1 -Port 6443`. 2. `x509: certificate is valid for …, not …` → the server URL in the Windows kubeconfig isn't `https://127.0.0.1:6443`. Copy it again from WSL. 3. An old `kubectl.exe` still runs → `Get-Command kubectl -All`. Remove the other path entry (Docker Desktop, Chocolatey or winget shims).

CKA corner: Step 2

- **Domain:** Cluster Architecture (kubeconfig, contexts, RBAC basics).
- **Practise:** `kubectl config view --minify` · `kubectl config use-context halo` · `kubectl config set-context --current --namespace=kube-system` · `kubectl auth can-i '*' '*' --all-namespaces` · `kubectl api-resources --namespaced=false`
- **Drill:** `kubectl config set-context halo-dev --cluster=halo --user=halo --namespace=dev` → use it → delete it with `kubectl config delete-context halo-dev`.

Interview angle: "Same pinned, checksum-verified toolchain on every workstation and runner, one named context per cluster, cluster-admin kubeconfigs ACL'd, and the GitOps CLI going through the API server instead of a public endpoint." **Phoenix angle:** you can run `kubectl`/`k9s` against Phoenix from Windows during market hours without opening a WSL shell.

Step 3: Verify the cluster end to end (B4)

Goal: prove scheduling, Services, DNS, storage and ingress from a Windows browser, then prove it all survives `wsl --shutdown`. **Why:** smoke-test every layer the later labs depend on *before* adding GitOps, so any later failure is about Argo CD and not the platform. These objects are throwaway, so imperative commands are fine. From Step 5 on, Argo CD is the only thing that applies app manifests. **Where:** [WSL] unless marked

#028 WSL Ubuntu shell (bash) - copy & paste

```
# 1) control plane + system pods
kubectl get nodes
kubectl get pods -A -o wide
kubectl top node                                  # metrics-server works (may need ~60s after boot)

# 2) a throwaway deployment + service + ingress (imperative on purpose — CKA style)
kubectl create namespace smoke
kubectl -n smoke create deployment hello --image=ghcr.io/stefanprodan/podinfo:6.17.0 --replicas=2 --port=9898
kubectl -n smoke expose deployment hello --port=80 --target-port=9898
kubectl -n smoke create ingress hello --class=traefik --rule="hello.halo.localhost/*=hello:80"
kubectl -n smoke rollout status deploy/hello
kubectl -n smoke get deploy,svc,ingress,endpointslices

# 3) DNS from inside a pod
kubectl -n smoke run dnscheck --image=busybox:1.37.0 --rm -it --restart=Never -- nslookup hello.smoke.svc.cluster.local

# 4) a PVC on local-path, written by a pod
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data, namespace: smoke }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: local-path
  resources: { requests: { storage: 1Gi } }
---
apiVersion: v1
kind: Pod
metadata: { name: writer, namespace: smoke }
spec:
```

```
  containers:
    - name: w
      image: busybox:1.37.0
      command: ["sh","-c","date > /data/hello.txt && sleep 3600"]
      volumeMounts: [{ name: d, mountPath: /data }]
  volumes: [{ name: d, persistentVolumeClaim: { claimName: data } }]
EOF
kubectl -n smoke wait --for=condition=Ready pod/writer --timeout=120s
kubectl -n smoke get pvc,pv
sudo ls /var/lib/rancher/k3s/storage/ && sudo cat /var/lib/rancher/k3s/storage/*_smoke_data/hello.txt
curl -s -H 'Host: hello.halo.localhost' http://127.0.0.1/version
```

Expected: the ingress `ADDRESS` is `10.255.255.10`; the PVC is `Bound` (it was `Pending` until the pod started, because local-path uses `WaitForFirstConsumer`); the host folder holds `hello.txt`; curl returns JSON with `"version": "6.17.0"`.

From [WIN] and [BROWSER]:

#029 Windows PowerShell - copy & paste

```
curl.exe -s http://hello.halo.localhost/version
```

Open **http://hello.halo.localhost** in Edge or Chrome and you should see the podinfo UI. Browsers send `*.localhost` to 127.0.0.1 themselves, so there's no hosts file and no DNS.

Clean up:

#030 WSL Ubuntu shell (bash) - copy & paste

```
kubectl delete namespace smoke
```

3.1 Restart test ([WIN])

#031 Windows PowerShell - copy & paste

```
wsl --shutdown
Start-Sleep 10
Start-ScheduledTask -TaskName 'WSL-halo-keepalive'     # same thing that happens at logon
Start-Sleep 60
kubectl get nodes; kubectl get pods -A | Select-String -NotMatch 'Running|Completed'
```

Expected: `halo Ready`. Only the header line appears in the second output (every pod is Running or Completed). The first boot after a shutdown takes about 30–90 s.

- ☐ Browser shows podinfo at `hello.halo.localhost`
- ☐ PVC bound and the file is visible under `/var/lib/rancher/k3s/storage`
- ☐ Cluster back by itself after `wsl --shutdown` + keep-alive task

If it fails 1. Browser 404 → Traefik is up but no rule matched. Check `kubectl get ingress -A` for the host spelling and `ingressClassName: traefik`. Appendix **T7**. 2. Browser "connection refused" → nothing reachable on Windows :80. In WSL run `sudo ss -ltnp | grep ':80 '`; from Windows run `Test-NetConnection 127.0.0.1 -Port 80`. Appendix **T7**. 3. PVC stays `Pending` → there's no pod using it yet (expected), or `kubectl -n kube-system logs deploy/local-path-provisioner` shows an error.

CKA corner: Step 3

- **Domain:** Services & Networking (20%), Storage (10%), Workloads (15%).
- **Practise:** `k get endpointslices -n smoke` · `k describe ingress hello -n smoke` · `k get sc local-path -o yaml | grep -E 'provisioner|volumeBindingMode|reclaimPolicy'` · `k run tmp --image=busybox:1.37.0 --rm -it --restart=Never -- wget -qO- hello.smoke` · `k get events -n smoke --sort-by=.lastTimestamp`
- **Drill:** generate (don't apply) a Deployment + Service + Ingress YAML trio with `$do` in under 3 minutes: `k create deploy web --image=nginx:1.29 --replicas=3 $do > web.yaml; k expose -f web.yaml --port=80 $do >> web.yaml` (add `---` between documents), then `k create ingress web --rule="web.halo.localhost/*=web:80" $do`.

Interview angle: "Before layering GitOps I smoke-test every primitive (scheduling, DNS, storage binding, ingress) and the restart path, so platform faults and delivery faults never get confused." **Phoenix angle:** you've just seen exactly where a Phoenix PVC lives on disk and that it survives a restart. **Call-out:** the `local-path` StorageClass uses `reclaimPolicy: Delete`, so deleting a PVC deletes the data. Before Timescale moves in, add a `local-path-retain` StorageClass (Retain) and scheduled `pg_dump` backups.

Block B4 done when: ☐ both shells talk to `halo` · ☐ the browser reaches an ingress · ☐ restart verified.

Step 4: Monorepo on GitHub + GitOps layout (block B5)

Goal: a private `ai-platform-portfolio` repo, cloned on WSL ext4, with the full skeleton, ADR-0001, the k3s host config captured as code, and the Argo CD folder layout ready for `halo` now and `aws` later. **Why:** in GitOps, the repo *is* the platform's desired state. Argo CD only reconciles what's in it. Getting the folder contract right on day 1 means ten labs and two environments can be added without moving anything. The repo goes public in January, so secret hygiene (`.gitignore`, no keys, noreply e-mail) starts with the first commit. **Where:** [GITHUB] then [WSL]

4.1 Create the repo and your Git identity

[GITHUB] **github.com → New repository** - Name `ai-platform-portfolio`, **Private**, **no** README / .gitignore / license (you'll push the first commit). - *Settings → Emails → "Keep my email addresses private"* is on. Copy your `ID+username@users.noreply.github.com` address.

[WSL] your *personal* push key (separate from Argo CD's read-only deploy key later):

#032 WSL Ubuntu shell (bash) - copy & paste

```
ssh-keygen -t ed25519 -C "halo-wsl-$(date +%Y%m)" -f ~/.ssh/id_ed25519_github -N ''
cat >> ~/.ssh/config <<'EOF'
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_github
  IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
cat ~/.ssh/id_ed25519_github.pub
```

[GITHUB] *Settings → SSH and GPG keys → New SSH key*, title `halo-wsl`, paste the `.pub` line.

#033 WSL Ubuntu shell (bash) - copy & paste

```
ssh -T git@github.com            # "Hi nikhilgs817! You've successfully authenticated..."
git config --global user.name  "Nikhil G S"
git config --global user.email "ID+nikhilgs817@users.noreply.github.com"   # your noreply address
git config --global init.defaultBranch main
git config --global core.autocrlf input       # never commit CRLF from Windows editors
git config --global pull.rebase true

mkdir -p ~/src && cd ~/src
git clone git@github.com:nikhilgs817/ai-platform-portfolio.git
cd ai-platform-portfolio
code .        # VS Code opens in "WSL: halo" mode
```

> **Different GitHub username?** After creating all the files below, run: `grep -rl nikhilgs817 . --exclude-dir=.git | xargs sed -i 's/nikhilgs817/<your-user>/g'`

4.2 Skeleton, placeholders, `.gitignore`, `.gitattributes`

[WSL] (inside `~/src/ai-platform-portfolio`)

#034 WSL Ubuntu shell (bash) - copy & paste

```
# ---- directories with README placeholders (dir|purpose) ----
while IFS='|' read -r d purpose; do
  mkdir -p "$d"
  [ -f "$d/README.md" ] || printf '# %s\n\n%s\n\n_Status: placeholder._\n' "$(basename "$d")" "$purpose" > "$d/README.md"
done <<'EOF'
platform|The paved road: everything that runs the platform itself.
platform/terraform|AWS GPU node groups, SageMaker, networking (later labs). No state files in Git.
platform/kubernetes|GitOps tree reconciled by Argo CD. See layout below.
platform/k3s|Host-level config for k3s nodes (config.yaml, systemd units). Applied by install scripts, not Argo CD.
platform/helm-charts|Reusable charts, starting with the FastAPI service template (L1 week 2).
platform/jenkins-shared-library|Jenkins pipeline library for build, scan, sign, push (L1 week 2+).
shared|Shared AI services used by several projects.
shared/llm-gateway|LiteLLM routing, budgets, fallbacks in front of self-hosted models (L2).
shared/rag-service|pgvector RAG that cites or abstains (L8 news RAG).
```

```
shared/mcp-servers|Read-only MCP tools (Phoenix data, docs) for agents (L2).
shared/eval-harness|Golden sets and CI eval gates for prompts, RAG and agents.
shared/observability|Prometheus, Loki, Tempo, OTel, Langfuse, OpenCost configs.
projects|Tenants that run on the platform.
projects/command-center|Phoenix Command Center: FastAPI core + UI, Timescale, Redis (first tenant).
projects/incident-resolution|AI-assisted incident triage (synthetic incidents only).
projects/release-command-center|Release readiness and change risk dashboard.
projects/code-review-agent|Self-hosted code review agent (pattern for the build-failure analyzer).
projects/sagemaker-stock-pipeline|Stock research ML platform: Feast, MLflow, Prefect, Evidently, SageMaker (L8).
docs|Documentation.
docs/architecture|Diagrams and platform overviews.
docs/adrs|Architecture Decision Records (one decision per file, numbered).
docs/runbooks|Operational procedures with measured results.
docs/demos|Demo scripts and recordings for interviews.
EOF

# ---- .gitignore: secrets and junk never reach Git ----
cat > .gitignore <<'EOF'
# ---- secrets & credentials (repo becomes PUBLIC in Jan 2027) ----
*.pem
*.key
*.p12
*.pfx
id_rsa*
id_ed25519*
*deploy*key*
!**/*deploy*key*.md
kubeconfig*
*.kubeconfig
.kube/
.env
.env.*
!.env.example
*secret*.yaml
!**/sealed-secret*.yaml
*.age
*.gpg
credentials*
```

```
# ---- terraform ----
.terraform/
*.tfstate
*.tfstate.*
*.tfvars
!*.tfvars.example
crash.log
# ---- backups / disk images / model weights ----
*.tar
*.tar.gz
*.vhdx
*.gguf
*.safetensors
# ---- python / node / editors ----
__pycache__/
*.py[cod]
.venv/
venv/
.pytest_cache/
.mypy_cache/
node_modules/
.idea/
.vscode/*
!.vscode/extensions.json
.DS_Store
Thumbs.db
EOF

# ---- .gitattributes: LF everywhere (shell scripts with CRLF break in WSL) ----
cat > .gitattributes <<'EOF'
* text=auto eol=lf
*.ps1 text eol=crlf
*.png binary
*.jpg binary
EOF
```

Root README:

#035 WSL Ubuntu shell (bash) - copy & paste

```
cat > README.md <<'EOF'
# ai-platform-portfolio

A self-hosted **AI platform on Kubernetes**: GitOps delivery, a supply-chain-secured paved road,
self-hosted LLM serving (no per-token API bills), RAG with evals, observability and FinOps, and MLOps.
Built and operated as if it were production, on one home-lab box (`halo`) plus AWS for GPU work.

| Lab | What it proves | Status |
|---|---|---|
| L1 | Kubernetes AI deployment platform: k3s, Argo CD, Kustomize, Harbor, Trivy, Syft, cosign, Kyverno, Rollouts | (in progress) week 1: GitOps |
| L2 | Self-hosted agents: llama.cpp on Radeon, LiteLLM, read-only MCP tools | planned |
| L8 | Stock research ML platform: Feast, MLflow, Prefect, Evidently, SageMaker, RAG | planned |

**Ground rules:** synthetic or public data only · AI is read-only with a human in the loop ·
no secrets in Git · every version pinned · Argo CD is the only thing that applies app manifests.

Start at [`docs/adrs`](docs/adrs) and [`platform/kubernetes`](platform/kubernetes).
EOF
```

4.3 ADR-0001

#036 WSL Ubuntu shell (bash) - copy & paste

```
cat > docs/adrs/0001-gitops-with-argocd-on-k3s-in-wsl2.md <<'EOF'
# ADR-0001: GitOps with Argo CD on k3s in WSL2

- **Status:** Accepted
- **Date:** 2026-11-02
- **Deciders:** Nikhil (platform owner)

## Context
The platform must host ten labs plus a real tenant (Phoenix: FastAPI, Timescale, Redis) around the clock
on a Windows 11 PC (Ryzen AI Max+ 395, 128 GB unified memory) that is also a daily driver, and later
extend to AWS GPU nodes. The end goal is to self-host several LLMs and cut per-token API spend. Goals:
production-like operations, CKA-relevant skills, reproducible rebuilds, nothing secret in a repo that
becomes public.
```

```
## Decision
1. **Runtime:** k3s v1.36 (pinned) as a native systemd service in an Ubuntu 24.04 WSL2 distro named `halo`.
   Embedded etcd, secrets encryption at rest, kubelet reservations. Node IP pinned to a dummy interface
   (10.255.255.10) so WSL's changing NAT IP never affects etcd or certificates.
2. **Networking:** WSL NAT mode + localhost forwarding. Traefik (bundled) runs with hostNetwork on :80/:443.
   Hostnames `*.halo.localhost` (browsers resolve these to loopback, no DNS needed).
3. **Delivery:** Argo CD (Helm chart, pinned) is the only actor that applies workloads. App of apps:
   one root Application per cluster (`clusters/<env>`), AppProjects `platform` and `lab`, Kustomize
   `base` + `overlays/<env>` per app. Automated sync with prune and selfHeal.
4. **Access:** Argo CD reads the private repo with a read-only SSH deploy key held only in the cluster
   and the password manager. Argo CD admin password lives only in the password manager.

## Alternatives considered
| Option | Why not (now) |
|---|---|
| k3d / kind | k3s inside Docker: extra runtime and network layer for long-lived workloads; good for CI throwaway clusters |
| Rancher Desktop / Docker Desktop Kubernetes | Hides systemd, certs, etcd: the things production and the CKA care about |
| kubeadm on Hyper-V VMs | Closest to the CKA exam, but heavier to run 24/7 on a daily PC; used separately for exam practice |
| Flux CD | Equally valid GitOps engine; Argo CD chosen for its UI (demo value), AppProjects (multi-tenancy) and Rollouts ecosystem |
| WSL mirrored networking | Node IP follows the active Windows adapter (Wi-Fi/VPN changes); NAT plus a pinned IP is more predictable |
| SQLite datastore (k3s default) | Fine for one node, but no etcd skills; etcd is safe here because the node IP is pinned |

## Consequences
- **+** Rebuild from Git in under an hour (k3s install script + Argo CD bootstrap); drift auto-reverted; audit trail = Git history.
- **+** The same repo layout extends to `aws` with a new root app and overlays, no restructuring.
- **–** Single node: no HA; node or disk loss means restoring from backups (wsl export, etcd snapshots, app backups).
- **–** WSL lifecycle (Windows updates, logon) affects uptime; mitigated by idle timeouts set to -1, a logon task and power settings.
- **–** No TLS in week 1 (`server.insecure`); acceptable on loopback only. Revisit with cert-manager.
- **–** GPU access from pods under WSL is non-standard (/dev/dxg, no device plugin); L2 starts with llama.cpp native on Windows as an external backend.
EOF
```

4.4 Capture the k3s host config as code (`platform/k3s/halo/`)

Why: the k3s config, Traefik override and systemd units are *host* config. Argo CD can't apply them because they have to exist before the cluster does. Keeping them in Git, with an install script, makes the rebuild drill (Step 9) a two-command job. It's the Ansible-role equivalent,

without needing Ansible yet.

#037 WSL Ubuntu shell (bash) - copy & paste

```
D=platform/k3s/halo; mkdir -p "$D"
WINHOME=$(wslpath "$(/mnt/c/Windows/System32/cmd.exe /c 'echo %USERPROFILE%' 2>/dev/null | tr -d '\r')")
sudo cp /etc/rancher/k3s/config.yaml                                "$D/config.yaml"
sudo cp /var/lib/rancher/k3s/server/manifests/traefik-config.yaml   "$D/traefik-config.yaml"
cp /etc/systemd/system/halo-node-ip.service                         "$D/halo-node-ip.service"
cp /etc/systemd/system/k3s.service.d/10-halo-node-ip.conf           "$D/10-halo-node-ip.conf"
cp /etc/modules-load.d/k3s.conf                                     "$D/modules-k3s.conf"
cp /etc/sysctl.d/90-k3s.conf                                        "$D/sysctl-90-k3s.conf"
cp /etc/wsl.conf                                                    "$D/wsl.conf"
cp "$WINHOME/.wslconfig"                                            "$D/wslconfig.ini"
sudo chown "$USER:" "$D"/*
```

`platform/k3s/halo/install-k3s.sh`, idempotent:

#038 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/k3s/halo/install-k3s.sh <<'EOF'
#!/usr/bin/env bash
# platform/k3s/halo/install-k3s.sh — install/reinstall k3s on node "halo" from the files in this folder.
# Usage (inside WSL, from repo root):  sudo platform/k3s/halo/install-k3s.sh
set -euo pipefail
K3S_VERSION="v1.36.5+k3s1"          # <- the ONE place the k3s version is pinned
DIR="$(cd "$(dirname "$0")" && pwd)"
[ "$(id -u)" -eq 0 ] || { echo "Run with sudo"; exit 1; }

echo ">> host prerequisites"
install -D -m 0644 "$DIR/modules-k3s.conf"       /etc/modules-load.d/k3s.conf
install -D -m 0644 "$DIR/sysctl-90-k3s.conf"     /etc/sysctl.d/90-k3s.conf
systemctl restart systemd-modules-load
sysctl --system >/dev/null

echo ">> stable node IP (halo0)"
install -D -m 0644 "$DIR/halo-node-ip.service"   /etc/systemd/system/halo-node-ip.service
install -D -m 0644 "$DIR/10-halo-node-ip.conf"   /etc/systemd/system/k3s.service.d/10-halo-node-ip.conf
systemctl daemon-reload
systemctl enable --now halo-node-ip.service
```

```
echo ">> k3s config + Traefik override"
install -D -m 0600 "$DIR/config.yaml"            /etc/rancher/k3s/config.yaml
install -D -m 0600 "$DIR/traefik-config.yaml"    /var/lib/rancher/k3s/server/manifests/traefik-config.yaml

echo ">> k3s ${K3S_VERSION}"
curl -fsSL -o /tmp/k3s-install.sh "https://raw.githubusercontent.com/k3s-io/k3s/${K3S_VERSION/+/%2B}/install.sh"
INSTALL_K3S_VERSION="$K3S_VERSION" sh /tmp/k3s-install.sh

echo ">> waiting for node Ready"
for i in $(seq 1 60); do
  k3s kubectl get node halo 2>/dev/null | grep -q ' Ready' && break; sleep 5
done
k3s kubectl get nodes -o wide
echo ">> done. Next: refresh your kubeconfig (see docs/runbooks/rebuild-halo.md)"
EOF
chmod +x platform/k3s/halo/install-k3s.sh

cat > platform/k3s/halo/README.md <<'EOF'
# platform/k3s/halo — host config for node "halo" (k3s in WSL2)

Applied by `install-k3s.sh` (not by Argo CD: these must exist before the cluster does).

| File | Installed to | Purpose |
|---|---|---|
| `config.yaml` | `/etc/rancher/k3s/config.yaml` | k3s server config (node IP, SANs, etcd, kubelet reservations) |
| `traefik-config.yaml` | `/var/lib/rancher/k3s/server/manifests/` | HelmChartConfig: Traefik hostNetwork :80/:443 |
| `halo-node-ip.service` | `/etc/systemd/system/` | dummy NIC `halo0` = 10.255.255.10 (stable node IP) |
| `10-halo-node-ip.conf` | `/etc/systemd/system/k3s.service.d/` | k3s starts after halo0 |
| `modules-k3s.conf`, `sysctl-90-k3s.conf` | `/etc/modules-load.d/`, `/etc/sysctl.d/` | kernel prerequisites |
| `wsl.conf` | `/etc/wsl.conf` (manual) | systemd, hostname, PATH hygiene |
| `wslconfig.ini` | `C:\Users\<you>\.wslconfig` (manual) | VM size, NAT, idle timeouts |
EOF
```

4.5 The GitOps layout (`platform/kubernetes/`)

Recommended layout, and why:

Output / reference - not a command

```
platform/kubernetes/
├── README.md
├── argocd/
│   ├── install/                    # Helm values for Argo CD itself
│   │   ├── values.yaml             #   common to every cluster
│   │   └── values-halo.yaml        #   per-cluster overrides (domain, sizing)
│   ├── bootstrap/                  # the ONLY manifests ever applied by hand (kubectl apply)
│   │   └── root-halo.yaml          #   root Application for cluster "halo"
│   ├── projects/                   # AppProjects (guardrails), managed by Argo CD after bootstrap
│   │   ├── kustomization.yaml
│   │   ├── platform.yaml
│   │   └── lab.yaml
│   └── clusters/                   # one folder per cluster = what the root app syncs
│       ├── halo/
│       │   ├── kustomization.yaml  #   list of child Applications on halo
│       │   ├── projects.yaml       #   child app → argocd/projects
│       │   └── podinfo.yaml        #   child app → apps/podinfo/overlays/halo   (Step 7)
│       └── aws/                    #   placeholder: root-aws will point here later
└── apps/                           # workloads, one folder per app
    └── podinfo/                    # (Step 7)
        ├── base/                   #   environment-neutral manifests
        └── overlays/
            ├── halo/               #   this box: replicas, host, quota, image tag
            └── aws/                #   placeholder
```

- `clusters/<env>` **holds Application manifests only.** It answers "what runs on this cluster?" in one file listing. Adding `aws` later means `bootstrap/root-aws.yaml` plus `clusters/aws/*.yaml` pointing at `apps/*/overlays/aws`.
- `apps/<name>/overlays/<env>` **holds the per-environment differences.** It answers "how does it differ there?" The base never knows about environments.
- `bootstrap/` **is tiny on purpose.** Everything else is reconciled, so a rebuild is `helm install` plus one `kubectl apply`.
- **Explicit Applications now, ApplicationSet later.** When you reach about 10 apps × 2 clusters, one ApplicationSet with a Git directory generator can replace the hand-written files in `clusters/<env>`. The folder contract stays the same.

> **Jenkins analogy:** `clusters/halo/kustomization.yaml` is the *seed job* (which pipelines exist). Each `Application` is a pipeline that never stops running. `overlays/halo` is that environment's parameter file.

Now create the files.

#039 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p platform/kubernetes/argocd/{install,bootstrap,projects,clusters/halo,clusters/aws} platform/kubernetes/apps
```

`platform/kubernetes/argocd/install/values.yaml`

#040 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/argocd/install/values.yaml <<'EOF'
# platform/kubernetes/argocd/install/values.yaml
# Common Argo CD values for every cluster. Chart argo/argo-cd 10.11.6 (Argo CD v3.5.6).
# NO secrets here: admin password is set via CLI and stored in the password manager.
global:
  domain: argocd.example.invalid      # overridden per cluster (values-<env>.yaml)
  revisionHistoryLimit: 3

configs:
  params:
    server.insecure: true             # TLS terminates at the ingress; no TLS in week 1 (loopback only)
  cm:
    admin.enabled: true               # local admin until SSO/OIDC (production: false)
    exec.enabled: false               # no web terminal into pods
    statusbadge.enabled: false
    timeout.reconciliation: 180s      # Git poll interval (+ jitter); prod uses webhooks
  rbac:
    policy.default: role:readonly     # any future SSO user is read-only unless granted more
  secret:
    createSecret: true                # chart creates argocd-secret; password is NOT set here

dex:
  enabled: false                      # no SSO yet

notifications:
  enabled: false                      # later: alert on sync failures / degraded health
```

```
controller:
  replicas: 1
  resources:
    requests: { cpu: 250m, memory: 512Mi }
    limits:   { memory: 2Gi }

repoServer:
  replicas: 1
  resources:
    requests: { cpu: 100m, memory: 256Mi }
    limits:   { memory: 1Gi }

server:
  replicas: 1
  resources:
    requests: { cpu: 50m, memory: 128Mi }
    limits:   { memory: 512Mi }
  ingress:
    enabled: true
    ingressClassName: traefik         # hostname defaults to global.domain
    tls: false

applicationSet:
  replicas: 1                         # kept for multi-cluster generators later
  resources:
    requests: { cpu: 25m, memory: 64Mi }
    limits:   { memory: 256Mi }

redis:
  resources:
    requests: { cpu: 50m, memory: 64Mi }
    limits:   { memory: 256Mi }
EOF
```

`platform/kubernetes/argocd/install/values-halo.yaml`

#041 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/argocd/install/values-halo.yaml <<'EOF'
# platform/kubernetes/argocd/install/values-halo.yaml — overrides for cluster "halo"
global:
  domain: argocd.halo.localhost       # browser resolves *.localhost to 127.0.0.1
EOF
```

`platform/kubernetes/argocd/projects/*`

#042 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/argocd/projects/platform.yaml <<'EOF'
# platform/kubernetes/argocd/projects/platform.yaml
# Platform-owned components: root apps, Argo CD itself, cluster add-ons. Full cluster scope.
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: platform
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-10"
    argocd.argoproj.io/sync-options: Prune=false   # never auto-delete a project
spec:
  description: Platform components owned by the platform team
  sourceRepos:
    - git@github.com:nikhilgs817/ai-platform-portfolio.git
    - https://argoproj.github.io/argo-helm          # Step 10: Argo CD manages its own chart
  destinations:
    - server: https://kubernetes.default.svc
      namespace: '*'
  clusterResourceWhitelist:
    - group: '*'
      kind: '*'
EOF

cat > platform/kubernetes/argocd/projects/lab.yaml <<'EOF'
# platform/kubernetes/argocd/projects/lab.yaml
```

```
# Tenant workloads (samples, Phoenix, LLM services). Least privilege: listed namespaces only,
# no cluster-scoped resources except the Namespace that CreateNamespace=true needs.
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: lab
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-10"
    argocd.argoproj.io/sync-options: Prune=false
spec:
  description: Tenant workloads on the halo/aws platform
  sourceRepos:
    - git@github.com:nikhilgs817/ai-platform-portfolio.git
  destinations:                                  # add one entry per tenant namespace, via PR
    - server: https://kubernetes.default.svc
      namespace: podinfo
  clusterResourceWhitelist:
    - group: ''
      kind: Namespace
  orphanedResources:
    warn: true                                   # flag anything in tenant namespaces not in Git
EOF

cat > platform/kubernetes/argocd/projects/kustomization.yaml <<'EOF'
# platform/kubernetes/argocd/projects/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - platform.yaml
  - lab.yaml
EOF
```

platform/kubernetes/argocd/bootstrap/root-halo.yaml

#043 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/argocd/bootstrap/root-halo.yaml <<'EOF'
# platform/kubernetes/argocd/bootstrap/root-halo.yaml
```

```
# The root ("app of apps") for cluster halo. Applied ONCE by hand at bootstrap; everything below it is GitOps.
# Deliberately NO resources-finalizer: deleting the root must never cascade-delete every app.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root-halo
  namespace: argocd
  labels:
    lab.platform/env: halo
spec:
  project: platform
  source:
    repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
    targetRevision: main
    path: platform/kubernetes/argocd/clusters/halo
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
    retry:
      limit: 5
      backoff: { duration: 10s, factor: 2, maxDuration: 3m }
EOF
```

`platform/kubernetes/argocd/clusters/halo/*`

#044 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/argocd/clusters/halo/projects.yaml <<'EOF'
# platform/kubernetes/argocd/clusters/halo/projects.yaml — Argo CD manages its own AppProjects
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: argocd-projects
```

```
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-10"            # projects before the apps that use them
spec:
  project: platform
  source:
    repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
    targetRevision: main
    path: platform/kubernetes/argocd/projects
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
EOF

cat > platform/kubernetes/argocd/clusters/halo/kustomization.yaml <<'EOF'
# platform/kubernetes/argocd/clusters/halo/kustomization.yaml
# Everything that runs on cluster "halo" = the child Applications listed here.
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - projects.yaml
  # - podinfo.yaml        # Step 7
  # - argocd.yaml         # Step 10 (optional)
EOF

cat > platform/kubernetes/argocd/clusters/aws/README.md <<'EOF'
# clusters/aws — placeholder

Future root: `bootstrap/root-aws.yaml` → this folder. Child Applications here point at
`apps/<name>/overlays/aws`. Intentionally empty until the AWS GPU labs.
EOF

cat > platform/kubernetes/apps/README.md <<'EOF'
# apps — one folder per workload
```

```
`<app>/base` is environment-neutral; `<app>/overlays/<env>` (halo, aws) carries replicas, hosts, quotas
and image tags. Argo CD child Applications in `argocd/clusters/<env>/` point at the overlays.
EOF
```

`platform/kubernetes/README.md`

#045 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/README.md <<'EOF'
# platform/kubernetes — the GitOps tree

Argo CD reconciles this folder. Humans change it only through Git.

| Path | Applied by | Purpose |
|---|---|---|
| `argocd/install/` | `helm upgrade --install` (bootstrap) or Argo CD itself (Step 10) | Argo CD Helm values |
| `argocd/bootstrap/root-<env>.yaml` | `kubectl apply` once per cluster | root Application (app of apps) |
| `argocd/projects/` | Argo CD (`argocd-projects` app) | AppProjects `platform`, `lab` |
| `argocd/clusters/<env>/` | Argo CD (root app) | child Applications for that cluster |
| `apps/<app>/base`, `apps/<app>/overlays/<env>` | Argo CD (child apps) | workloads via Kustomize |

Environments: `halo` (home lab, k3s in WSL2) and `aws` (later, GPU).
EOF
```

4.6 Check for secrets, commit, push

#046 WSL Ubuntu shell (bash) - copy & paste

```
git add -A
git status --short
# Nothing private should match:
git grep -nIE 'BEGIN [A-Z ]*PRIVATE KEY|client-key-data|token:|password:' -- . ':!*.md' || echo "secret scan: clean"
git commit -m "feat(L1): monorepo skeleton, ADR-0001, k3s host config, Argo CD GitOps layout"
git push -u origin main
```

Expected: `secret scan: clean`, then `* [new branch] main -> main`. **Verify:** [GITHUB] refresh the repo and you should see the tree above and the README rendered.

- ☐ Repo private, first commit on `main`
- ☐ `platform/k3s/halo/` holds config.yaml, traefik-config.yaml, units, install script
- ☐ ADR-0001 present

If it fails 1. `Permission denied (publickey)` → `ssh -vT git@github.com` and check that `~/.ssh/config` points at `id_ed25519_github` and the key is added to your *account* (not as a deploy key). 2. The secret scan matches `password:` in a YAML comment → reword the comment. Never commit and then try to "fix it later". Git history is forever once the repo is public. 3. Scripts fail later with `$'\r': command not found` → the file was saved with CRLF. Check `.gitattributes`, then run `git add --renormalize . && git commit`.

CKA corner: Step 4

- **Domain:** Workloads & Scheduling (manifests, Kustomize is built into kubectl).
- **Practise:** `kubectl kustomize platform/kubernetes/argocd/projects` · `kubectl apply -k <dir> --dry-run=server` · `kubectl explain appproject.spec --recursive | head -40` (once Argo CD is installed) · `kubectl diff -k <dir>`
- **Drill:** `kubectl create configmap app-cfg --from-literal=MODE=lab $do > cm.yaml`, then write a `kustomization.yaml` that includes it and adds `namespace: dev` + label `env=dev`; render it with `kubectl kustomize .`.

Interview angle: "I designed the repo contract first: clusters list Applications, apps carry base plus environment overlays, and only a one-file bootstrap is ever applied by hand. A second region or cluster is additive, not a refactor." **Phoenix angle:** Phoenix gets `apps/command-center/{base,overlays/halo}` and one line in `clusters/halo`. Its source code stays in `projects/command-center/`, and its deployment is reviewed like any other change.

Steps 5–7: Argo CD, app of apps, podinfo (blocks B6–B7)

Step 5: Install Argo CD with Helm (B6)

5.1 Helm install from the committed values

Goal: Argo CD v3.5.6 (chart 10.11.6) in namespace `argocd`, configured only by the values files in Git. **Why:** Argo CD can't install itself on an empty cluster (chicken and egg), so the very first install is a Helm release, exactly as in production bootstraps. Using the values files from the repo means the same command reproduces the same Argo CD on any cluster. In Jenkins terms, this installs the controller from a JCasC file.
Where: [WSL] (repo root)

#047 WSL Ubuntu shell (bash) - copy & paste

```
cd ~/src/ai-platform-portfolio
kubectl config current-context            # must print: halo
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update argo
helm search repo argo/argo-cd --version 10.11.6     # CHART VERSION 10.11.6, APP VERSION v3.5.6

helm upgrade --install argocd argo/argo-cd \
  --version 10.11.6 \
  --namespace argocd --create-namespace \
  -f platform/kubernetes/argocd/install/values.yaml \
  -f platform/kubernetes/argocd/install/values-halo.yaml \
  --wait --timeout 10m
```

Expected: `Release "argocd" does not exist. Installing it now.` … `STATUS: deployed` … `REVISION: 1`, followed by chart notes.

Verify:

#048 WSL Ubuntu shell (bash) - copy & paste

```
kubectl -n argocd get pods
kubectl -n argocd get ingress
curl -s -o /dev/null -w '%{http_code}\n' -H 'Host: argocd.halo.localhost' http://127.0.0.1/
helm -n argocd list
```

Expected: pods `argocd-application-controller-0`, `-applicationset-controller-…`, `-redis-…`, `-repo-server-…`, `-server-…` all `Running`; ingress `argocd-server` with host `argocd.halo.localhost` and address `10.255.255.10`; HTTP `200`.

[BROWSER] Open **http://argocd.halo.localhost**. The Argo CD login page should appear. If it doesn't, the fallback is a port-forward: `kubectl -n argocd port-forward svc/argocd-server 8080:80`, then open http://localhost:8080.

5.2 Log in, rotate the admin password, delete the bootstrap secret

Why: the chart generates a random first password and stores it in `argocd-initial-admin-secret`. Production practice is to rotate it straight away, keep the real one only in a password manager, and delete the bootstrap secret so it can't leak through a `kubectl get secret`. The CLI goes through a port-forward (`ARGOCD_OPTS`, set in Step 2), so there's no DNS or TLS dependency. **Where:** [WSL]

#049 WSL Ubuntu shell (bash) - copy & paste

```
echo "$ARGOCD_OPTS"                                 # --port-forward --port-forward-namespace argocd --plaintext
INITIAL=$(argocd admin initial-password -n argocd | head -1)
argocd login --username admin --password "$INITIAL"
# 1) Generate a strong password IN YOUR PASSWORD MANAGER (entry: "Argo CD admin — halo"), then:
argocd account update-password --current-password "$INITIAL"     # prompts for the new password twice
unset INITIAL
argocd login --username admin                       # prompts; use the new password
kubectl -n argocd delete secret argocd-initial-admin-secret
argocd account get-user-info
```

Expected: `'admin:login' logged in successfully` … `Password updated` … `secret "argocd-initial-admin-secret" deleted` … `Logged In: true, Username: admin`.

[BROWSER] Log in to the UI with `admin` and the new password. On Windows ([WIN]) the CLI works the same way: `argocd login --username admin`.

5.3 Connect the private repo with a read-only deploy key

Why: a *deploy key* is an SSH key that grants access to **one repo**, and with "Allow write access" unticked it's read-only. If Argo CD is ever compromised, the attacker can read one repo and can't push. The private key goes into the cluster as a Secret (encrypted at rest, thanks to `secrets-encryption`) and into your password manager, and never into Git. `argocd repo add` simply creates a labelled Secret in `argocd`. Step 9 creates that same Secret with `kubectl`. **Where:** [WSL] then [GITHUB] then [WSL]

#050 WSL Ubuntu shell (bash) - copy & paste

```
ssh-keygen -t ed25519 -C "argocd-halo-readonly" -f ~/.ssh/argocd-halo-deploy -N ''
cat ~/.ssh/argocd-halo-deploy.pub
# Also store the PRIVATE key (~/.ssh/argocd-halo-deploy) in the password manager entry "Argo CD deploy key — halo"
```

[GITHUB] repo → **Settings → Deploy keys → Add deploy key**. Title `argocd-halo (read-only)`, paste the `.pub` line, **leave "Allow write access" unticked**, then **Add key**.

#051 WSL Ubuntu shell (bash) - copy & paste

```
argocd repo add git@github.com:nikhilgs817/ai-platform-portfolio.git \
  --name ai-platform-portfolio \
  --ssh-private-key-path ~/.ssh/argocd-halo-deploy
argocd repo list
kubectl -n argocd get secrets -l argocd.argoproj.io/secret-type=repository
```

Expected: `Repository 'git@github.com:nikhilgs817/ai-platform-portfolio.git' added`. `argocd repo list` shows `TYPE git … STATUS Successful`. One repository Secret exists. In the UI, *Settings → Repositories* shows a green *Successful*.

> GitHub's SSH host keys come preloaded in `argocd-ssh-known-hosts-cm`, so you don't need `--insecure-ignore-host-key`. Never use that flag.

- ☐ UI reachable at `argocd.halo.localhost`
- ☐ Admin password rotated and in the password manager; initial secret deleted
- ☐ Repo connected (Successful) with a read-only deploy key; private key only in ~/.ssh and the password manager

If it fails 1. `helm … context deadline exceeded` → `kubectl -n argocd get pods` and describe the pending pod. Usually images are still pulling on a slow link. Re-run the same `helm upgrade --install` command (it's idempotent). 2. `argocd login` hangs → the port-forward needs a Running `argocd-server` pod and a working kubeconfig. Run `kubectl -n argocd get pods`, then `argocd login --username admin --port-forward --port-forward-namespace argocd --plaintext` explicitly. 3. Repo status `Failed … ssh: handshake failed` → the deploy key isn't on *this* repo, or you pasted the private key in GitHub. Appendix **T6**.

CKA corner: Step 5

- **Domain:** Cluster Architecture (Helm and Kustomize are in the 2025+ curriculum; CRDs and operators).
- **Practise:** `helm -n argocd list` · `helm -n argocd get values argocd` · `helm template argocd argo/argo-cd --version 10.11.6 -f platform/kubernetes/argocd/install/values.yaml | grep -c '^kind:'` · `kubectl get crd | grep argoproj` · `kubectl -n argocd get secret -l argocd.argoproj.io/secret-type=repository -o yaml | head -20`
- **Drill:** `kubectl create secret generic demo --from-literal=user=lab $do` → decode one value with `kubectl get secret demo -o jsonpath='{.data.user}' | base64 -d`, and explain why base64 isn't encryption (that's what `secrets-encryption` is for).

Interview angle: "Argo CD itself is installed from version-pinned, Git-tracked Helm values. Bootstrap credentials are rotated and destroyed on day 0, and Git access is a single-repo, read-only key that never touches the repo." **Phoenix angle:** Phoenix deploys need nothing on the box except Argo CD's read-only key. The trading code never gets cluster credentials.

Step 6: AppProjects + the root Application (app of apps) (B7)

Goal: apply the two AppProjects and `root-halo` once. After that, Argo CD manages everything else, including the projects themselves. **Why:** an **AppProject** is a guardrail: which repos, which namespaces and which kinds of resources a group of apps may touch. Think of it as the Jenkins folder-level permissions plus allowed credentials. `lab` is least privilege (named namespaces only, no cluster objects except Namespace). `platform` is broad, for platform components. The **root Application** points at `clusters/halo`, a folder that contains only *other Application manifests*. One root, many children: "app of apps". `automated` + `prune` + `selfHeal` means Git is enforced, not just suggested.
Where: [WSL]

#052 WSL Ubuntu shell (bash) - copy & paste

```
cd ~/src/ai-platform-portfolio
kubectl apply -k platform/kubernetes/argocd/projects          # the two AppProjects (adopted by Argo CD next)
kubectl apply -f platform/kubernetes/argocd/bootstrap/root-halo.yaml
```

Expected:

Output / reference - not a command

```
appproject.argoproj.io/lab created
appproject.argoproj.io/platform created
application.argoproj.io/root-halo created
```

Verify (give it about 30 s):

#053 WSL Ubuntu shell (bash) - copy & paste

```
argocd proj list
argocd app list
argocd app get root-halo
```

Expected: projects `default`, `lab`, `platform`. Apps `root-halo` and `argocd-projects` both `Synced` / `Healthy`. In the UI, `root-halo` shows a tree with one child Application, `argocd-projects`, which owns the two AppProjects.

What the sync options mean

Setting	Effect	Production note
`automated`	Applies every new commit on `main` without a click	Keep for apps; some orgs use manual sync for prod with automated PR promotion
`prune: true`	Deletes live objects that were removed from Git	Protected here: AppProjects have `Prune=false`
`selfHeal: true`	Reverts manual `kubectl` changes within seconds	Makes Git the only change path; break-glass = disable selfHeal in Git
`CreateNamespace=true`	Argo CD creates the destination namespace	Labels set via `managedNamespaceMetadata` (Step 7)

Setting	Effect	Production note
no finalizer on root	Deleting `root-halo` leaves children running	Children *do* have the finalizer, so removing one from Git cleans up its resources

- ☐ `root-halo` and `argocd-projects` Synced/Healthy
- ☐ `lab` project restricts destinations to `podinfo`

If it fails 1. `root-halo` shows `ComparisonError … repository not accessible` → the repo URL in `root-halo.yaml` must match `argocd repo list` **exactly** (`git@github.com:…`). Appendix **T6**. 2. `application destination … is not permitted in project` → the destination namespace isn't in the AppProject. Fix it in Git (`projects/*.yaml`), then push. 3. Stuck `OutOfSync` → appendix **T5** (`argocd app diff root-halo`).

Step 7: Sample app podinfo: base + halo overlay, deployed only through Git (B7)

Goal: podinfo at **http://podinfo.halo.localhost**, deployed only by a Git push. **Why podinfo:** it's built for exactly this. It has `/healthz` and `/readyz` probes, a UI that shows **version and hostname** (you can *see* replicas and rollouts), Prometheus metrics, and it runs non-root with a read-only filesystem, so it passes the **restricted** Pod Security level that production namespaces should enforce. It also gives you a believable stand-in for a FastAPI service before the Phoenix template exists. **Where:** [WSL]

7.1 Base (`apps/podinfo/base/`)

#054 WSL Ubuntu shell (bash) - copy & paste

```
B=platform/kubernetes/apps/podinfo/base; mkdir -p "$B"

cat > $B/deployment.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo
spec:
  replicas: 1
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 120          # a bad rollout turns Degraded after 2 min, not 10
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }   # old pods keep serving if the new ones fail
  selector:
    matchLabels:
```

```
    app.kubernetes.io/name: podinfo
template:
  metadata:
    labels:
      app.kubernetes.io/name: podinfo
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "9797"
  spec:
    automountServiceAccountToken: false
    securityContext:
      runAsNonRoot: true
      runAsUser: 10001
      runAsGroup: 10001
      fsGroup: 10001
      seccompProfile: { type: RuntimeDefault }
    containers:
      - name: podinfo
        image: ghcr.io/stefanprodan/podinfo:6.17.0
        imagePullPolicy: IfNotPresent
        command:
          - ./podinfo
          - --port=9898
          - --port-metrics=9797
          - --level=info
          - --data-path=/data
        ports:
          - { name: http, containerPort: 9898, protocol: TCP }
          - { name: http-metrics, containerPort: 9797, protocol: TCP }
        readinessProbe:
          httpGet: { path: /readyz, port: http }
          initialDelaySeconds: 2
          periodSeconds: 5
        livenessProbe:
          httpGet: { path: /healthz, port: http }
          initialDelaySeconds: 5
          periodSeconds: 10
        resources:
```

```
            requests: { cpu: 25m, memory: 32Mi }
            limits:   { memory: 128Mi }         # no CPU limit: avoid throttling; requests drive scheduling
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: [ALL] }
          volumeMounts:
            - { name: data, mountPath: /data }
      volumes:
        - name: data
          emptyDir: { sizeLimit: 64Mi }
EOF

cat > $B/service.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/base/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: podinfo
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: podinfo
  ports:
    - { name: http, port: 80, targetPort: http, protocol: TCP }
EOF

cat > $B/ingress.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/base/ingress.yaml — host is set per environment in the overlay
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: podinfo
spec:
  ingressClassName: traefik
  rules:
    - host: podinfo.example.invalid
      http:
```

```
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: podinfo
                port: { name: http }
EOF

cat > $B/pdb.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/base/pdb.yaml — keeps 1 pod during voluntary disruptions (kubectl drain)
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: podinfo
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: podinfo
EOF

cat > $B/limitrange.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/base/limitrange.yaml — defaults for any container that forgets resources
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
spec:
  limits:
    - type: Container
      defaultRequest: { cpu: 50m, memory: 64Mi }
      default:        { memory: 256Mi }
      max:            { memory: 1Gi }
EOF

cat > $B/kustomization.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/base/kustomization.yaml
```

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
labels:
  - pairs:
      app.kubernetes.io/part-of: lab-samples
      app.kubernetes.io/managed-by: argocd
    includeSelectors: false              # never change selectors (immutable on Deployments)
resources:
  - deployment.yaml
  - service.yaml
  - ingress.yaml
  - pdb.yaml
  - limitrange.yaml
EOF
```

7.2 Overlay `halo` (and the `aws` placeholder)

#055 WSL Ubuntu shell (bash) - copy & paste

```
O=platform/kubernetes/apps/podinfo/overlays; mkdir -p $O/halo $O/aws

cat > $O/halo/kustomization.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/overlays/halo/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: podinfo
resources:
  - ../../base
  - resourcequota.yaml
labels:
  - pairs:
      lab.platform/env: halo
    includeSelectors: false
images:
  - name: ghcr.io/stefanprodan/podinfo
    newTag: "6.17.0"                     # Step 8d breaks this on purpose
patches:
  - path: patch-deployment.yaml
  - target: { kind: Ingress, name: podinfo }
```

```
    patch: |-
      - op: replace
        path: /spec/rules/0/host
        value: podinfo.halo.localhost
EOF

cat > $O/halo/patch-deployment.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/overlays/halo/patch-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo
spec:
  replicas: 2                           # Step 8a changes this to 3
  template:
    spec:
      containers:
        - name: podinfo
          env:
            - name: PODINFO_UI_MESSAGE
              value: "halo - deployed by Argo CD from Git"
            - name: PODINFO_UI_COLOR
              value: "#2d6a4f"
EOF

cat > $O/halo/resourcequota.yaml <<'EOF'
# platform/kubernetes/apps/podinfo/overlays/halo/resourcequota.yaml — tenant budget on halo
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-budget
spec:
  hard:
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.memory: 2Gi
    pods: "10"
    persistentvolumeclaims: "2"
```

```
    services.loadbalancers: "0"          # tenants use the shared ingress, not host ports
EOF

cat > $O/aws/README.md <<'EOF'
# overlays/aws — placeholder (AWS GPU labs). Intentionally no kustomization.yaml yet.
EOF

kubectl kustomize $O/halo | grep -E '^kind:|image:|host:|replicas:|namespace:' | sort | uniq -c
```

Expected render check: kinds `Deployment, Service, Ingress, PodDisruptionBudget, LimitRange, ResourceQuota`; `image: ghcr.io/stefanprodan/podinfo:6.17.0`; `host: podinfo.halo.localhost`; `replicas: 2`; `namespace: podinfo`.

7.3 The child Application, then deploy by pushing

#056 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/argocd/clusters/halo/podinfo.yaml <<'EOF'
# platform/kubernetes/argocd/clusters/halo/podinfo.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: podinfo
  namespace: argocd
  labels:
    lab.platform/env: halo
    lab.platform/tenant: podinfo
  annotations:
    argocd.argoproj.io/sync-wave: "10"
  finalizers:
    - resources-finalizer.argocd.argoproj.io   # removing this app from Git also removes its resources
spec:
  project: lab
  source:
    repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
    targetRevision: main
    path: platform/kubernetes/apps/podinfo/overlays/halo
  destination:
    server: https://kubernetes.default.svc
    namespace: podinfo
```

```
  revisionHistoryLimit: 10
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - PruneLast=true
    managedNamespaceMetadata:
      labels:
        pod-security.kubernetes.io/enforce: restricted
        pod-security.kubernetes.io/enforce-version: latest
        pod-security.kubernetes.io/warn: restricted
        lab.platform/tenant: podinfo
        lab.platform/env: halo
    retry:
      limit: 5
      backoff: { duration: 10s, factor: 2, maxDuration: 3m }
EOF

sed -i 's|^  # - podinfo.yaml .*|  - podinfo.yaml|' platform/kubernetes/argocd/clusters/halo/kustomization.yaml
cat platform/kubernetes/argocd/clusters/halo/kustomization.yaml

git add -A
git commit -m "feat(podinfo): base + halo overlay, child Application in lab project"
git push
argocd app get root-halo --refresh >/dev/null      # don't wait for the 3-min poll
until argocd app get podinfo >/dev/null 2>&1; do sleep 5; done   # child app appears within ~10 s
argocd app wait podinfo --sync --health --timeout 300
```

Expected: `podinfo` appears under `root-halo` and ends `Synced` / `Healthy`.

Verify:

#057 WSL Ubuntu shell (bash) - copy & paste

```
argocd app get podinfo
kubectl get ns podinfo --show-labels                    # PSA restricted + tenant labels
kubectl -n podinfo get deploy,rs,pods,svc,ingress,pdb,quota,limitrange
kubectl -n podinfo describe quota tenant-budget | tail -8
curl -s -H 'Host: podinfo.halo.localhost' http://127.0.0.1/version
```

[WIN] `curl.exe -s http://podinfo.halo.localhost/version` → `{"commit":"…","version":"6.17.0"}` [BROWSER] **http://podinfo.halo.localhost**: green page with "halo - deployed by Argo CD from Git". Refresh a few times and the hostname switches between the 2 pods.

- ☐ podinfo Synced/Healthy, created **only** by `git push`
- ☐ Namespace enforces PSA `restricted`; quota and LimitRange present
- ☐ UI reachable from the Windows browser

Checkpoint reached: Argo CD is syncing a sample app from Git.

If it fails 1. Pods rejected with `violates PodSecurity "restricted:latest"` → a securityContext field is missing. The event tells you which one. Fix it in `base/deployment.yaml` and push. 2. `ImagePullBackOff` → appendix **T4** (tag typo, rate limit, DNS from the node). 3. Ingress 404 in the browser but `curl -H Host:` works in WSL → host spelling (`podinfo.halo.localhost`), or the browser is using a proxy. Appendix **T7**.

CKA corner: Steps 6–7

- **Domain:** Workloads & Scheduling (15%): Deployments, rollouts, probes, resource requests, quotas, PDBs. Also Services & Networking (Ingress).
- **Practise:** `k -n podinfo rollout history deploy/podinfo` · `k -n podinfo get pods -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image,NODE:.spec.nodeName` · `k -n podinfo describe quota` · `k auth can-i create deployments -n podinfo --as=system:serviceaccount:podinfo:default` · `k get ns podinfo -o jsonpath='{.metadata.labels}'`
- **Drill:** in a scratch namespace, `k create quota q --hard=pods=2,requests.cpu=500m -n scratch`, then `k create deploy x --image=nginx:1.29 --replicas=3 -n scratch` and find why only 2 pods exist (`k describe rs -n scratch`). Delete the namespace afterwards.

Interview angle: "Tenants get a namespace with restricted Pod Security, a quota and default limits, all declared next to the app and created by Argo CD under a least-privilege AppProject. Onboarding a team is one PR." **Phoenix angle:** Phoenix Command Center follows this exact template: restricted PSA, probes for "survives any pod kill", quota so a runaway backtest can't starve the live service, and a PDB for safe node maintenance.

Steps 8–10: Prove GitOps, rebuild drill, Argo CD manages itself (blocks B8–B10)

Keep two windows open for all of Step 8: - **Window A ([WSL]):** `watch -n2 'kubectl -n podinfo get deploy,rs,pods -o wide; echo; argocd app list'` - **Window B ([BROWSER]):** the Argo CD UI on the `podinfo` app (tree view), plus a tab on `http://podinfo.halo.localhost`

Step 8: Prove GitOps works (B8)

8a: Change replicas in Git, push, watch it sync

Why: proves the main GitOps loop: *commit → Argo CD notices → cluster converges*. Nobody runs `kubectl apply`. **Where:** [WSL] (repo root)

#058 WSL Ubuntu shell (bash) - copy & paste

```
cd ~/src/ai-platform-portfolio
sed -i 's/^  replicas: 2 .*/  replicas: 3                          # Step 8a/' platform/kubernetes/apps/podinfo/overlays/halo/patch-deployment.yaml
git diff --stat && git commit -am "podinfo(halo): scale to 3 replicas" && git push
argocd app get podinfo --refresh | grep -E 'Sync Status|Health Status'
```

Expected: within seconds (thanks to `--refresh`; otherwise within about 3–4 min by polling), a third pod appears and the app returns to `Synced` / `Healthy` at the new commit SHA. The browser hostname now rotates across 3 pods. **Verify:** `kubectl -n podinfo get deploy podinfo -o jsonpath='{.spec.replicas}{"\n"}'` → `3` - ☐ 8a done

> Production uses a GitHub **webhook** to Argo CD so syncs start instantly. That needs a reachable endpoint; the 180 s poll is the lab equivalent.

8b: `kubectl delete` the Deployment, watch Argo CD bring it back

Why: proves *self-heal for deletions*. The cluster is a cache of Git, not the source of truth.

#059 WSL Ubuntu shell (bash) - copy & paste

```
kubectl -n podinfo delete deployment podinfo
sleep 15; kubectl -n podinfo get deploy podinfo
argocd app get podinfo | grep -E 'Sync Status|Health Status'
```

Expected: the Deployment is recreated within about 5–15 s (`AGE` a few seconds). The app goes briefly `OutOfSync` / `Missing`, then back to `Synced` / `Healthy`. The UI's *Events* show the auto-sync. - ☐ 8b done

8c: `kubectl scale` by hand, watch the drift get reverted

Why: proves *self-heal for drift*. In a large organisation, this is what stops a 2 a.m. hand change from quietly becoming permanent.

#060 WSL Ubuntu shell (bash) - copy & paste

```
kubectl -n podinfo scale deployment podinfo --replicas=5
sleep 10; kubectl -n podinfo get deploy podinfo
```

Expected: replicas jump to 5, then within seconds Argo CD sets them back to **3**, and the extra pods terminate. - ☐ 8c done

> Later, with an HPA, you'll tell Argo CD to ignore `spec.replicas` (`ignoreDifferences`). Otherwise Argo CD and the HPA fight.

8d: Push a bad change (wrong image tag), see Degraded, fix with `git revert`

Why: shows what failure looks like and why rollback is a **Git operation**. With `maxUnavailable: 0` the old pods keep serving, so users never notice. `kubectl rollout undo` would be undone by selfHeal, and `argocd app rollback` is blocked while auto-sync is on. The supported rollback is `git revert`.

#061 WSL Ubuntu shell (bash) - copy & paste

```
sed -i 's/newTag: "6.17.0" .*/newTag: "6.99.0"                    # Step 8d: does not exist/' platform/kubernetes/apps/podinfo/overlays/halo/kustomization.yaml
git commit -am "podinfo(halo): bump to 6.99.0 (intentionally broken)" && git push
argocd app get podinfo --refresh >/dev/null
sleep 30; kubectl -n podinfo get pods
curl -s -H 'Host: podinfo.halo.localhost' http://127.0.0.1/version     # still 6.17.0: old pods serve
sleep 100; argocd app get podinfo | grep -E 'Sync Status|Health Status'
```

Expected: one new pod shows `ErrImagePull` → `ImagePullBackOff`, the 3 old pods stay `Running`, and the app is **Synced** but **Degraded** after `progressDeadlineSeconds` (120 s). `kubectl -n podinfo describe deploy podinfo | grep -A2 Progressing` shows `ProgressDeadlineExceeded`.

Fix it:

#062 WSL Ubuntu shell (bash) - copy & paste

```
git revert --no-edit HEAD && git push
argocd app get podinfo --refresh >/dev/null
argocd app wait podinfo --health --timeout 180
```

Expected: the broken ReplicaSet scales to 0 and the app returns to `Healthy`. Git history now shows the bad commit *and* its revert, which is your audit trail. - ☐ 8d done (Degraded seen, recovered by revert)

8e: Find the sync history in the UI and the CLI

#063 WSL Ubuntu shell (bash) - copy & paste

```
argocd app history podinfo
kubectl -n argocd get application podinfo -o jsonpath='{range .status.history[*]}{.id}{"  "}{.revision}{"  "}{.deployedAt}{"\n"}{end}'
git log --oneline -6
```

[BROWSER] UI: app `podinfo` → **History and Rollback**. Each entry is a deployed Git SHA with a timestamp, matching `git log`.

Expected: at least 4 entries (initial, scale to 3, broken 6.99.0, revert). The SHAs match `git log`. - ☐ 8e done

8f: `wsl --shutdown`, start again, confirm self-recovery

Where: [WIN]

#064 Windows PowerShell - copy & paste

```
wsl --shutdown
Start-Sleep 10
Start-ScheduledTask -TaskName 'WSL-halo-keepalive'
Start-Sleep 90
kubectl get nodes
kubectl -n argocd get pods
argocd app list
curl.exe -s http://podinfo.halo.localhost/version
```

Expected: node `Ready`, all Argo CD pods `Running`, `root-halo`, `argocd-projects` and `podinfo` all `Synced` / `Healthy`, and curl returns `6.17.0`. You didn't touch anything. If you have time, also do a real Windows restart and check the same after logging in. - ☐ 8f done

CKA corner: Step 8

- **Domain:** Troubleshooting (30%), the biggest domain. Also Workloads (rollouts).
- **Practise:** `k -n podinfo describe pod <failing>` (read *Events*) · `k -n podinfo get events --sort-by=.lastTimestamp | tail` · `k -n podinfo rollout status deploy/podinfo --timeout=30s` · `k -n podinfo get rs -o wide` · `k -n podinfo logs deploy/podinfo --tail=20`
- **Drill (no Argo CD, in a scratch namespace):** `k create deploy web --image=nginx:1.29 -n scratch` → `k set image deploy/web nginx=nginx:9.9.9 -n scratch` → diagnose → `k rollout undo deploy/web -n scratch` → `k rollout history deploy/web -n scratch`. On the exam, `rollout undo` *is* the answer. Under GitOps it's `git revert`. Know both.

Interview angle: "I can show a live demo: a manual delete, a manual scale and a bad image push, each reverted or contained automatically, with rollback done as a Git revert and the full deployment history tied to commit SHAs." **Phoenix angle:** a bad Phoenix release can't take down the running Command Center (old pods keep serving), and recovery is one `git revert` from your phone's GitHub app.

Step 9: Rebuild drill (B9)

Goal: destroy the lab and prove that **Git + two secrets** (deploy key, admin password) bring everything back. Measure the time (your RTO).
Why: a backup you haven't restored isn't a backup. This drill tests the whole chain: WSL snapshot → host config from Git → k3s → Argo CD bootstrap → apps from Git. It's a DR test, the VMware "restore from template and re-run the playbooks" exercise.

9.1 Add the two bootstrap scripts to the repo ([WSL])

#065 WSL Ubuntu shell (bash) - copy & paste

```
cd ~/src/ai-platform-portfolio

cat > platform/k3s/halo/install-clients.sh <<'EOF'
#!/usr/bin/env bash
# platform/k3s/halo/install-clients.sh — pinned kubectl/helm/argocd/k9s in WSL + kubeconfig "halo" (+ copy to Windows)
# Usage (as your user, NOT sudo): platform/k3s/halo/install-clients.sh
set -euo pipefail
KV=v1.36.6; HV=v4.3.1; AV=v3.5.6; K9V=v0.51.0
T=$(mktemp -d); trap 'rm -rf "$T"' EXIT; cd "$T"
curl -fsSLO "https://dl.k8s.io/release/${KV}/bin/linux/amd64/kubectl"
curl -fsSLO "https://dl.k8s.io/release/${KV}/bin/linux/amd64/kubectl.sha256"
echo "$(cat kubectl.sha256)  kubectl" | sha256sum --check --quiet
sudo rm -f /usr/local/bin/kubectl && sudo install -m 0755 kubectl /usr/local/bin/kubectl
curl -fsSLO "https://get.helm.sh/helm-${HV}-linux-amd64.tar.gz"
curl -fsSLO "https://get.helm.sh/helm-${HV}-linux-amd64.tar.gz.sha256sum"
sha256sum --check --quiet "helm-${HV}-linux-amd64.tar.gz.sha256sum"
tar xzf "helm-${HV}-linux-amd64.tar.gz" && sudo install -m 0755 linux-amd64/helm /usr/local/bin/helm
curl -fsSLO "https://github.com/argoproj/argo-cd/releases/download/${AV}/argocd-linux-amd64"
curl -fsSLO "https://github.com/argoproj/argo-cd/releases/download/${AV}/cli_checksums.txt"
grep ' argocd-linux-amd64$' cli_checksums.txt | sha256sum --check --quiet
sudo install -m 0755 argocd-linux-amd64 /usr/local/bin/argocd
curl -fsSLO "https://github.com/derailed/k9s/releases/download/${K9V}/k9s_Linux_amd64.tar.gz"
curl -fsSLO "https://github.com/derailed/k9s/releases/download/${K9V}/checksums.sha256"
grep ' k9s_Linux_amd64.tar.gz$' checksums.sha256 | sha256sum --check --quiet
tar xzf k9s_Linux_amd64.tar.gz k9s && sudo install -m 0755 k9s /usr/local/bin/k9s

# kubeconfig: context/cluster/user all named "halo"
mkdir -p ~/.kube
sudo cat /etc/rancher/k3s/k3s.yaml | sed -e 's/: default$/: halo/' > ~/.kube/config
```

```
chmod 600 ~/.kube/config
WINHOME=$(wslpath "$(/mnt/c/Windows/System32/cmd.exe /c 'echo %USERPROFILE%' 2>/dev/null | tr -d '\r')")
mkdir -p "$WINHOME/.kube" && cp ~/.kube/config "$WINHOME/.kube/config"

# shell setup (idempotent)
if ! grep -q '# --- k8s tooling (CKA habits) ---' ~/.bashrc; then cat >> ~/.bashrc <<'RC'
# --- k8s tooling (CKA habits) ---
source <(kubectl completion bash)
source <(helm completion bash)
source <(argocd completion bash)
alias k=kubectl
complete -o default -F __start_kubectl k
export do="--dry-run=client -o yaml"
export now="--force --grace-period 0"
export ARGOCD_OPTS="--port-forward --port-forward-namespace argocd --plaintext"
RC
fi
kubectl version && helm version --short && argocd version --client --short
echo "Windows kubeconfig refreshed (new cluster CA). Open a new shell: source ~/.bashrc"
EOF
chmod +x platform/k3s/halo/install-clients.sh

cat > platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh <<'EOF'
#!/usr/bin/env bash
# platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh — install Argo CD and hand the cluster to GitOps.
# Usage: platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh halo
#        (deploy key expected at ~/.ssh/argocd-<env>-deploy; override with DEPLOY_KEY=...)
set -euo pipefail
ENV="${1:-halo}"
CHART_VERSION="10.11.6"                     # <- the ONE place the Argo CD chart version is pinned for bootstrap
REPO_URL="git@github.com:nikhilgs817/ai-platform-portfolio.git"
DEPLOY_KEY="${DEPLOY_KEY:-$HOME/.ssh/argocd-${ENV}-deploy}"
A="$(git rev-parse --show-toplevel)/platform/kubernetes/argocd"

[ "$(kubectl config current-context)" = "$ENV" ] || { echo "kube context must be '$ENV'"; exit 1; }
[ -f "$DEPLOY_KEY" ] || { echo "Missing $DEPLOY_KEY: restore it from the password manager"; exit 1; }

echo ">> Argo CD chart ${CHART_VERSION}"
```

```
helm repo add argo https://argoproj.github.io/argo-helm --force-update >/dev/null
helm repo update argo >/dev/null
helm upgrade --install argocd argo/argo-cd --version "$CHART_VERSION" \
  --namespace argocd --create-namespace \
  -f "$A/install/values.yaml" -f "$A/install/values-${ENV}.yaml" \
  --wait --timeout 10m

echo ">> repository credential (in-cluster Secret only; never in Git)"
kubectl -n argocd create secret generic repo-ai-platform-portfolio \
  --from-literal=type=git --from-literal=name=ai-platform-portfolio \
  --from-literal=url="$REPO_URL" --from-file=sshPrivateKey="$DEPLOY_KEY" \
  --dry-run=client -o yaml \
| kubectl label --local -f - argocd.argoproj.io/secret-type=repository -o yaml \
| kubectl apply -f -

echo ">> AppProjects + root application"
kubectl apply -k "$A/projects"
kubectl apply -f "$A/bootstrap/root-${ENV}.yaml"

cat <<MSG
>> Bootstrap complete. Now:
   1) argocd login --username admin --password "\$(argocd admin initial-password -n argocd | head -1)"
   2) argocd account update-password   (use the password-manager value)
   3) kubectl -n argocd delete secret argocd-initial-admin-secret
MSG
EOF
chmod +x platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh

git add -A && git commit -m "chore(halo): rebuild scripts for clients and Argo CD bootstrap" && git push
```

9.2 Run the drill: full DR from the baseline WSL export

> **Before you start:** the deploy private key and the Argo CD admin password are in your password manager (Step 5). Everything else has been pushed (`git status` is clean). If either is missing, stop and fix that first.

Start the clock and write down the time.

[WIN-ADMIN]: safety snapshot, then destroy and restore the baseline:

#066 Windows PowerShell - copy & paste

```
$bk = "$env:USERPROFILE\WSL-backups"
wsl --terminate halo
wsl --export halo "$bk\halo-predrill-$(Get-Date -Format yyyyMMdd-HHmm).tar"   # safety net, keep until the drill passes
wsl --unregister halo                                                         # destroys the distro and its vhdx
$base = (Get-ChildItem "$bk\halo-baseline-*.tar" | Sort-Object Name | Select-Object -Last 1).FullName
wsl --import halo "$env:LOCALAPPDATA\wsl\halo" $base
wsl --manage halo --set-sparse true
wsl --set-default halo
Start-ScheduledTask -TaskName 'WSL-halo-keepalive'
wsl -d halo -e sh -c 'hostname; whoami; ps -p1 -o comm='                      # halo, <you>, systemd
```

[WSL]: identity, repo, host, cluster, clients, Argo CD:

#067 WSL Ubuntu shell (bash) - copy & paste

```
# 1) GitHub push identity: generate a NEW key (rotation), add it on GitHub, then delete the old "halo-wsl" key there
ssh-keygen -t ed25519 -C "halo-wsl-$(date +%Y%m)" -f ~/.ssh/id_ed25519_github -N ''
printf 'Host github.com\n  HostName github.com\n  User git\n  IdentityFile ~/.ssh/id_ed25519_github\n  IdentitiesOnly yes\n' > ~/.ssh/config
chmod 600 ~/.ssh/config; cat ~/.ssh/id_ed25519_github.pub      # [GITHUB] add as SSH key, remove the old one
ssh -T git@github.com                                          # run AFTER adding the key on GitHub

# 2) Repo
sudo apt-get update -qq && sudo apt-get install -y -qq git curl jq unzip bash-completion
git config --global user.name "Nikhil G S"; git config --global user.email "ID+nikhilgs817@users.noreply.github.com"
git config --global core.autocrlf input; git config --global pull.rebase true
mkdir -p ~/src && cd ~/src && git clone git@github.com:nikhilgs817/ai-platform-portfolio.git && cd ai-platform-portfolio

# 3) Host config + k3s (from Git)
sudo cp platform/k3s/halo/wsl.conf /etc/wsl.conf      # already there from the baseline; keeps them identical
sudo platform/k3s/halo/install-k3s.sh

# 4) Clients + kubeconfig (also refreshes the Windows kubeconfig: the cluster CA is new)
platform/k3s/halo/install-clients.sh && source ~/.bashrc

# 5) Deploy key: paste the PRIVATE key from the password manager
install -m 600 /dev/null ~/.ssh/argocd-halo-deploy && nano ~/.ssh/argocd-halo-deploy

# 6) Argo CD bootstrap; the apps come back from Git with no further commands
```

```
platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh halo
until kubectl -n argocd get application podinfo >/dev/null 2>&1; do sleep 5; done   # child app created from Git

# 7) Credentials: log in with the generated password, set the password-manager value, delete the bootstrap secret
argocd login --username admin --password "$(argocd admin initial-password -n argocd | head -1)"
argocd account update-password
kubectl -n argocd delete secret argocd-initial-admin-secret
```

Then:

#068 WSL Ubuntu shell (bash) - copy & paste

```
argocd app wait podinfo --sync --health --timeout 600
argocd app list
curl -s -H 'Host: podinfo.halo.localhost' http://127.0.0.1/version
```

[WIN] `kubectl get nodes; curl.exe -s http://podinfo.halo.localhost/version`

Stop the clock. Expected: podinfo `Synced` / `Healthy` with 3 replicas and `6.17.0`, without you applying a single app manifest. Typical time: 35–60 min (most of it downloads and key handling).

Quick variant (k3s only, ~15 min): `sudo /usr/local/bin/k3s-uninstall.sh && sudo platform/k3s/halo/install-k3s.sh && platform/k3s/halo/install-clients.sh && platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh halo`. Do this one whenever you change k3s config.

If it fails 1. `wsl --import` says "distribution already exists" → the unregister didn't finish. Run `wsl -l -v`, then `wsl --unregister halo` again. 2. The bootstrap script fails at the repo Secret → the deploy key file has Windows line endings or missing header lines. Re-paste it and check that `head -1 ~/.ssh/argocd-halo-deploy` prints `-----BEGIN OPENSSH PRIVATE KEY-----`. 3. Windows kubectl shows `x509: certificate signed by unknown authority` → the Windows kubeconfig still has the old CA. Re-run `install-clients.sh`.

Once the drill passes, delete the safety `halo-predrill-*.tar` and take a **new** post-platform export (`halo-platform-YYYYMMDD.tar`), so you have two restore points.

9.3 Write the runbook ([WSL])

#069 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p docs/runbooks
cat > docs/runbooks/rebuild-halo.md <<'EOF'
# Runbook: rebuild cluster "halo" from Git

| | |
|---|---|
```

```
| Scope | k3s node `halo` (WSL2 distro on the MS-S1 Max) + Argo CD + all GitOps apps |
| RTO target | 60 min · **Measured: __ min (YYYY-MM-DD)** |
| RPO | Git: last push · stateful data: see "Not covered" |
| Needs | baseline WSL export · password manager entries "Argo CD admin — halo", "Argo CD deploy key — halo" · GitHub access |

## When to use
WSL distro corrupted or deleted · k3s/etcd unrecoverable · after major host changes (as a drill, monthly).

## Procedure A: full DR (WSL baseline import), ~45 min
1. Windows (admin): `wsl --terminate halo` → `wsl --export halo <bk>\halo-predrill-<ts>.tar` → `wsl --unregister halo`
2. `wsl --import halo $env:LOCALAPPDATA\wsl\halo <bk>\halo-baseline-<date>.tar` → `wsl --manage halo --set-sparse true`
   → `wsl --set-default halo` → `Start-ScheduledTask WSL-halo-keepalive`
3. WSL: new GitHub SSH key (rotate: remove the old key on GitHub) → clone repo to `~/src/ai-platform-portfolio`
4. `sudo platform/k3s/halo/install-k3s.sh`
5. `platform/k3s/halo/install-clients.sh` (also refreshes the Windows kubeconfig)
6. Restore `~/.ssh/argocd-halo-deploy` (mode 600) from the password manager
7. `platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh halo`
8. Rotate the admin password to the password-manager value; `kubectl -n argocd delete secret argocd-initial-admin-secret`

## Procedure B: k3s only, ~15 min
`sudo /usr/local/bin/k3s-uninstall.sh` → steps 4, 5, 7, 8 above.

## Procedure C: etcd point-in-time restore (cluster state only)
`sudo k3s etcd-snapshot ls` → `sudo systemctl stop k3s` →
`sudo k3s server --cluster-reset --cluster-reset-restore-path=<snapshot>` → `sudo systemctl start k3s`

## Verification
- [ ] `kubectl get nodes` → halo Ready (WSL and Windows)
- [ ] `argocd app list` → all apps Synced/Healthy
- [ ] `curl.exe http://podinfo.halo.localhost/version` → expected version
- [ ] `wsl --shutdown` + keep-alive task → recovers unattended

## Not covered (yet)
PVC data (local-path) is NOT restored by Git. Before Phoenix stores real state: Retain StorageClass,
scheduled pg_dump / Redis RDB to a backup location, and a restore test in this runbook.

## Drill log
| Date | Procedure | Duration | Issues found | Follow-up |
```

```
|---|---|---|---|---|
| YYYY-MM-DD | A | __ min | | |
EOF
git add -A && git commit -m "docs(runbook): rebuild-halo with measured drill results" && git push
```

Fill in the measured time and the issues you hit, then commit again.

- ☐ Drill completed from baseline export; podinfo returned from Git alone
- ☐ `docs/runbooks/rebuild-halo.md` committed with the real duration
- ☐ New post-platform export taken; personal GitHub SSH key rotated

CKA corner: Step 9

- **Domain:** Cluster Architecture, Installation & Configuration (etcd backup and restore is a near-certain exam task).
- **Practise:** `sudo k3s etcd-snapshot save --name pre-change` · `sudo k3s etcd-snapshot ls` · `sudo ls -l /var/lib/rancher/k3s/server/db/snapshots/` · `sudo k3s certificate check` · `kubectl get --raw /readyz/etcd`
- **Drill (kubeadm syntax, write from memory):** `ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd.db --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key`, then the matching `etcdutl snapshot restore /tmp/etcd.db --data-dir /var/lib/etcd-restore` and the manifest edit that points etcd at the new data dir.

Interview angle: "I run timed DR drills: I destroy the node, restore a base image, and the platform plus every app comes back from Git with two secrets. I documented the RTO and the gaps, such as stateful data, in a runbook." **Phoenix angle:** worst case for Phoenix is now "about 45 minutes to a fully deployed Command Center". The runbook's "Not covered" section is the backlog item to fix *before* Timescale holds data you care about.

Step 10 (optional, B10): Argo CD manages its own Helm install

Goal: an `argocd` Application that renders the same chart and values from the repo, so Argo CD upgrades are just Git commits too. **Why:** it closes the last imperative gap (`helm upgrade`), and Argo CD 3.x's annotation-based tracking avoids the old label conflict with Helm's `app.kubernetes.io/instance`. **Risks:** a bad values change can break the very controller that would fix it. Two owners (Helm CLI and Argo CD) must never both be used again. CRDs and the Redis-secret init job (a Helm hook) behave a bit differently under Argo CD. Mitigations: `prune: false`, `ServerSideApply`, test changes with `helm template` first, and keep the bootstrap script as the break-glass path.

#070 WSL Ubuntu shell (bash) - copy & paste

```
cat > platform/kubernetes/argocd/clusters/halo/argocd.yaml <<'EOF'
# platform/kubernetes/argocd/clusters/halo/argocd.yaml — Argo CD manages its own Helm release (same chart + values as bootstrap)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
```

```
  name: argocd
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-20"
  # NO finalizer: deleting this Application must never delete Argo CD itself
spec:
  project: platform
  sources:
    - repoURL: https://argoproj.github.io/argo-helm
      chart: argo-cd
      targetRevision: 10.11.6                # bump here = Argo CD upgrade (keep bootstrap script in sync)
      helm:
        releaseName: argocd                  # MUST match the bootstrap release name
        valueFiles:
          - $values/platform/kubernetes/argocd/install/values.yaml
          - $values/platform/kubernetes/argocd/install/values-halo.yaml
    - repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
      targetRevision: main
      ref: values
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  ignoreDifferences:
    - kind: Secret
      name: argocd-secret
      jsonPointers: [/data]                  # admin password + server key are set at runtime, not from Git
  syncPolicy:
    automated:
      prune: false                           # never auto-delete parts of the controller
      selfHeal: true
    syncOptions:
      - ServerSideApply=true                 # large CRDs
      - RespectIgnoreDifferences=true
EOF
sed -i 's|^  # - argocd.yaml .*|  - argocd.yaml|' platform/kubernetes/argocd/clusters/halo/kustomization.yaml
git add -A && git commit -m "feat(argocd): Argo CD manages its own Helm release" && git push
```

```
argocd app get root-halo --refresh >/dev/null; sleep 20
argocd app get argocd | head -20
```

Expected: app `argocd` `Synced` / `Healthy` with no pod restarts (the rendered manifests match what Helm installed). From now on, **never run `helm upgrade argocd` again** except to break glass. **Break glass:** if a bad commit breaks Argo CD, run `git revert`, then `helm upgrade --install …` from Step 5.1 (the bootstrap script) to force the last good state. - ☐ (optional) `argocd` app Synced/Healthy; ADR note added: "Argo CD is self-managed from chart 10.11.6"

Appendix: troubleshooting, glossary, final checklist, what's next

Troubleshooting

Start with the same triage every time ([WSL]):

#071 WSL Ubuntu shell (bash) - copy & paste

```
systemctl is-active halo-node-ip k3s; kubectl get nodes; kubectl get pods -A | grep -vE 'Running|Completed'; argocd app list
```

T1: WSL won't start, or systemd is off

Symptom	Likely cause	Fix
`wsl -d halo` errors `0x80370102` / `HCS_E_HYPERV_NOT_INSTALLED`	Virtual Machine Platform off, or SVM off in BIOS	`dism /online /enable-feature /featurename:VirtualMachinePlatform /all`, reboot; check SVM in BIOS
`ps -p1 -o comm=` prints `init`	`wsl.conf` `[boot] systemd=true` missing or typo, or no full restart	fix `/etc/wsl.conf`, then `wsl --shutdown` (closing the window isn't enough)
Hangs on start after a `.wslconfig` edit	Bad value (for example `memory=48G B`)	rename `.wslconfig`, run `wsl --shutdown`, start, fix the value
Distro gone from `wsl -l`	unregistered or profile reset	restore: `wsl --import halo <dir> <export.tar>` (runbook)

T2: k3s fails on cgroups

`journalctl -u k3s` shows `failed to find memory cgroup (v2)` or `cpuset` missing. 1. `stat -fc %T /sys/fs/cgroup` must be `cgroup2fs`. If it's `tmpfs` (hybrid), add under `[wsl2]` in `.wslconfig`: `kernelCommandLine = cgroup_no_v1=all systemd.unified_cgroup_hierarchy=1`, then `wsl --shutdown`. 2. `cat /sys/fs/cgroup/cgroup.controllers` must list `cpuset cpu io memory pids`. If not, run `wsl --update` (old kernel). 3. `sudo k3s check-config` lists the failing items.

T3: k3s fails on iptables / nftables

`journalctl -u k3s` shows `iptables: … Could not fetch rule set generation id` or flannel `failed to set up masquerade`. 1. Make sure `prefer-bundled-bin: true` is in `/etc/rancher/k3s/config.yaml`, then `sudo systemctl restart k3s`. 2. Host tools mixing legacy and nft: `sudo update-alternatives --set iptables /usr/sbin/iptables-nft && sudo update-alternatives --set ip6tables /usr/sbin/ip6tables-nft`. 3. `lsmod | grep br_netfilter` is empty → `sudo modprobe br_netfilter`, and check `/etc/modules-load.d/k3s.conf`.

T4: ImagePullBackOff

`kubectl -n <ns> describe pod <pod>` → read the last *Events*: | Message | Cause | Fix | |---|---|---| | `manifest unknown` / `not found` | tag typo (Step 8d on purpose) | fix the tag in the overlay → push (or `git revert`) | | `toomanyrequests` | Docker Hub anonymous rate limit | prefer ghcr.io/quay.io

images now; Harbor proxy-cache later | | `dial tcp: lookup … no such host` | node DNS broken (VPN, DNS tunneling) | in WSL: `resolvectl` / `cat /etc/resolv.conf`, `curl -I https://ghcr.io`; try `dnsTunneling=false`; `wsl --shutdown` | | `x509: certificate signed by unknown authority` | TLS-intercepting proxy/AV | add the proxy CA under `/usr/local/share/ca-certificates/`, `update-ca-certificates`, restart k3s |

T5: App stuck OutOfSync

1. `argocd app diff <app>` shows exactly which field differs.
2. A controller or webhook mutates a field (for example an HPA on `replicas`, or defaulted fields) → add `ignoreDifferences` for that path in the Application, in Git.
3. Someone keeps changing it by hand → selfHeal will revert it. Find out who with `kubectl get events` and the audit trail.
4. Kustomize build error → `argocd app get <app>` shows `ComparisonError`. Reproduce it with `kubectl kustomize <path>` locally.
5. Hard refresh: `argocd app get <app> --hard-refresh` (clears the repo-server cache).

T6: Repo fails to connect

`argocd repo list` STATUS `Failed`: 1. `ssh: handshake failed: ssh: unable to authenticate` → the deploy key is on another repo, or the *public* key was put in the Secret. Check with `kubectl -n argocd get secret repo-… -o jsonpath='{.data.sshPrivateKey}' | base64 -d | head -1`, which should print `-----BEGIN OPENSSH PRIVATE KEY-----`. 2. `knownhosts: key is unknown` → `argocd cert list --cert-type ssh | grep github`. Add the key with `ssh-keyscan github.com | argocd cert add-ssh --batch` (and confirm the fingerprint against GitHub's published one). 3. URL mismatch: Applications must use the identical URL (`git@github.com:nikhilgs817/ai-platform-portfolio.git`), not `https://` or `ssh://` variants.

T7: Ingress returns 404 (or refuses connection) from Windows

Test	Meaning
[WSL] `curl -sI -H 'Host: podinfo.halo.localhost' http://127.0.0.1/` → 200	cluster side is fine; problem is Windows ↔ WSL
[WSL] → **404**	Traefik reached, no matching rule: check `kubectl get ingress -A` (host, `ingressClassName: traefik`, Service name/port)
[WSL] → connection refused	Traefik not on :80: `sudo ss -ltnp | grep ':80 '`, `kubectl -n kube-system get pods -l app.kubernetes.io/name=traefik`, `kubectl -n kube-system logs job/helm-install-traefik`
[WIN] `Test-NetConnection 127.0.0.1 -Port 80` False	port not relayed: Windows already uses :80 (`Get-NetTCPConnection -LocalPort 80 -State Listen`), `localhostForwarding=false`, or relay stale → `wsl --shutdown` and start again
[WIN] works in curl.exe, not in browser	browser proxy/VPN extension, or browser tried `::1` first and gave up: try `http://127.0.0.1` with a Host-header extension, or disable the proxy for `*.localhost`

If port 80 on Windows can't be freed: move Traefik to 8080/8443 in `platform/k3s/halo/traefik-config.yaml` (`ports.web.port: 8080`, `ports.websecure.port: 8443`; you can then drop the root/NET_BIND_SERVICE override), re-run `install-k3s.sh`, and browse to `http://podinfo.halo.localhost:8080`.

T8: Cluster gone after a Windows reboot

1. `wsl -l -v` → `halo Stopped` → the keep-alive task didn't fire. Check *Task Scheduler → WSL-halo-keepalive → History / Last Run Result*, then `Start-ScheduledTask WSL-halo-keepalive`.
2. Running but `kubectl` times out → k3s still starting (30–90 s), or failed: `wsl -d halo -e sudo journalctl -u k3s -b --no-pager | tail -40`.
3. `halo0` missing → `systemctl status halo-node-ip`. The `dummy` module must load (`/etc/modules-load.d/k3s.conf`).
4. Stops again after about 1 min with no terminal → `.wslconfig` idle settings (`vmIdleTimeout=-1`, `instanceIdleTimeout=-1`) not applied: `wsl --version` (update), `wsl --shutdown`.
5. Reboot happened while logged out → *AtLogOn* didn't trigger. See the note in Step 0.6 about adding an *AtStartup* trigger.

T9: "The WSL IP changed"

With this design that's **expected and harmless**: `eth0` changes, the node stays on `10.255.255.10`, and Windows uses `127.0.0.1`. If something does break: 1. `kubectl get node halo -o wide` INTERNAL-IP isn't `10.255.255.10` → `node-ip` / `flannel-iface` missing from `config.yaml`, or k3s started before `halo0` existed (check the drop-in `10-halo-node-ip.conf`). 2. Windows kubeconfig points at a `172.x` address → copy it again (`install-clients.sh`). It must be `https://127.0.0.1:6443`. 3. Something on Windows needs the WSL IP (rare) → `wsl -d halo hostname -I`. Never hard-code it.

T10: Clock skew after sleep (`x509: certificate has expired or is not yet valid`)

`timedatectl` in WSL shows a wrong time → `sudo hwclock -s` or `sudo systemctl restart systemd-timesyncd`. Avoid sleep (Step 0.6).

Glossary

Term	Plain words (and the comparison you already know)
WSL2	A lightweight Hyper-V VM that runs real Linux kernels for "distros" (like a single-purpose ESXi VM managed by Windows)
Distro	One Linux root filesystem inside WSL (`halo`); export/import ≈ VM template
.wslconfig / wsl.conf	VM-level settings (vCPU/RAM/NIC) / per-distro settings (≈ cloud-init)
k3s	CNCF-certified Kubernetes in one binary; the same API as kubeadm clusters
etcd	The cluster's database: every object you `kubectl apply` lives here
containerd	The container runtime the kubelet asks to run containers
flannel	The pod network (VXLAN overlay) that gives every pod an IP
Traefik / Ingress	HTTP router: picks a Service by hostname/path (≈ IHS/F5 virtual host rules in front of WebSphere)
ServiceLB	k3s's simple `LoadBalancer` implementation using host ports
local-path	Storage provisioner: each PVC is a folder on the node's disk

Term	Plain words (and the comparison you already know)
PSA (Pod Security Admission)	Namespace label that rejects unsafe pods (`restricted` = non-root, no privilege escalation, ...)
ResourceQuota / LimitRange	Namespace budget / default per-container sizes
PDB	"Keep at least N pods" during voluntary disruptions such as drains
Helm	Package manager: chart + values → manifests (≈ a parameterised Ansible role)
Kustomize	Patch-based overlays on plain YAML (base + env differences), built into kubectl
GitOps	Git holds desired state; an in-cluster agent pulls and reconciles continuously (pull, not Jenkins-push)
Argo CD Application	"Sync this Git path to this cluster/namespace", a continuously running deploy pipeline
AppProject	Guardrails for a group of Applications: allowed repos, namespaces, resource kinds (≈ Jenkins folder + credentials scope)
App of apps	A root Application whose manifests are other Applications
Sync / Prune / Self-heal	Apply Git → delete what Git removed → undo manual drift
Sync wave	Ordering hint (lower first), for example projects before apps
Health: Progressing / Healthy / Degraded	Argo CD's view of whether resources actually work, not just whether they were applied
Deploy key	SSH key scoped to one GitHub repo, read-only if write is unticked
RTO / RPO	How long a restore takes / how much data you may lose

Final checklist: "Argo CD syncing a sample app from Git"

Foundation - ☐ WSL 3.0.x, distro `halo` (Ubuntu 24.04.5), systemd, hostname `halo`, 48 GB / 16 vCPU - ☐ `.wslconfig` NAT + localhost forwarding + idle timeouts −1; keep-alive task; no sleep on AC - ☐ Kernel prerequisites persistent (modules-load.d, sysctl.d); all preflight checks OK - ☐ Baseline export taken (and a post-platform export after Step 9)

Cluster - ☐ k3s v1.36.5+k3s1, node `halo` Ready on 10.255.255.10, embedded etcd, secrets encryption on, snapshots scheduled - ☐ Traefik on :80/:443 (hostNetwork); ServiceLB and local-path available - ☐ kubectl/helm/argocd pinned in WSL **and** Windows; Windows context `halo` → `https://127.0.0.1:6443` - ☐ Smoke tests passed (deployment, DNS, PVC, ingress from Windows browser); survives `wsl --shutdown`

GitOps - ☐ `ai-platform-portfolio` (private) with skeleton, `.gitignore`, `.gitattributes`, ADR-0001, `platform/k3s/halo/` - ☐ Argo CD chart 10.11.6 / v3.5.6 installed from committed values; UI at `http://argocd.halo.localhost` - ☐ Admin password rotated and in password manager; `argocd-initial-admin-secret` deleted - ☐ Repo connected with **read-only** deploy key; private key not in Git - ☐ AppProjects `platform` + `lab`; `root-halo` → `argocd-projects` + `podinfo`, all Synced/Healthy - ☐ **podinfo deployed only via `git push`, reachable at `http://podinfo.halo.localhost`**

Proofs - ☐ 8a replicas via Git · 8b delete healed · 8c scale reverted · 8d Degraded → `git revert` · 8e history in UI + CLI · 8f recovers after `wsl --shutdown` - ☐ Rebuild drill done; `docs/runbooks/rebuild-halo.md` with measured RTO - ☐ (optional) Argo CD self-managed

Hygiene - ☐ `git log -p | grep -E 'PRIVATE KEY|password'` finds nothing sensitive - ☐ Export tarballs stored off-Git, treated as secrets

---

What's next (out of scope this week)

L1, the rest of the paved road (weeks 2–4) - **Service template:** FastAPI + Dockerfile + Helm chart in `platform/helm-charts/`, Jenkinsfile using `platform/jenkins-shared-library/`. - **Supply chain:** Harbor (registry + proxy cache), Trivy (scan), Syft (SBOM), cosign (sign), **Kyverno** policy that blocks unsigned images in tenant namespaces. - **Progressive delivery:** Argo Rollouts canary for the template service. - **Platform services:** Sealed Secrets or External Secrets, cert-manager + TLS (then turn off `server.insecure`), the observability stack (Prometheus, Loki, Tempo, OTel, Grafana), OpenCost. - **Terraform** for AWS GPU node groups, plus the `aws` overlay and `root-aws`. - **Phoenix Command Center moves in:** `apps/command-center` with Timescale + Redis on a **Retain** StorageClass, backups, and the restore test added to the runbook.

Toward the end goal: self-hosted models instead of paying per token 1. **L2:** llama.cpp natively on Windows (Vulkan/HIP on the Radeon 8060S) as an external backend in k3s → **LiteLLM** in front (routing, per-tenant budgets, fallbacks, usage logs) → Phoenix agents (Morning Brief 08:55, Explain-This-Grade, EOD Journal 16:20) with read-only MCP tools. 2. **Several models side by side:** size by GPU memory (model weights + KV cache × concurrent requests). Keep small and fast models always loaded, load large ones on demand, and route by task in LiteLLM. 3. **Measure the savings:** Langfuse and OpenCost give cost per request and per tenant. Compare that with the equivalent API price per million tokens. That comparison is your interview story. 4. **AWS:** vLLM, plus Triton or NIM on GPU nodes with the NVIDIA device plugin, batching and VRAM sizing, all synced by the same Argo CD layout through the `aws` overlay.